Design and Evaluation of an Automated Three-Tier External Memory System for Long-Horizon Coding Agents

Final Capstone Proposal

Authors	Kaiqiang Zhang and Ruifeng Cui
Degree Program	Computer Science Master of Science
Course	GRAD 695 Research Methodology and Writing
Instructor	Abdel Ejnioui, Ph.D.
Institution	Harrisburg University of Science and Technology
Project Area	Techniques for Autonomous Software Development
Submission Date	September 26, 2026

A Capstone II-ready proposal in software engineering in the era of AI

AI Use Disclosure

OpenAI Codex was used as a writing and document-production assistant to organize previously completed weekly deliverables, compare the report against the instructor's proposal guide, draft transitions and explanatory prose, check terminology across chapters, create original diagrams from the approved design, format tables, and generate the final PDF. The authors selected the topic, developed the research questions, reviewed the literature, decided the architecture and evaluation design, and remain responsible for the final submission.

Academic references in this report come from the verified twenty-source literature set prepared during Weeks 5 through 7. Product documentation and software-factory examples were used only as implementation context and not as substitutes for scholarly evidence. No experimental results were fabricated. All thresholds, datasets, procedures, and expected contributions are identified as planned Capstone II work. The authors reviewed the report for factual consistency, proposal tense, source alignment, and scope before submission.

Abstract

Coding agents can inspect repositories, modify files, run tests, and repair failures, but individual model sessions do not preserve a reliable project state across days and tasks. Existing research demonstrates episodic reflection, hierarchical context management, repository memory, adaptive compression, and learned memory operations. These mechanisms address parts of the problem, while the complete coding-memory lifecycle remains fragmented. Users still decide what to save, when to update it, how to handle conflicts, and what to reload. This proposal presents an automated three-tier external-memory system for long-horizon coding agents. Project Memory will store stable validated knowledge, a Daily Log will preserve chronological evidence, and Session Memory will maintain resumable task state. Hooks, a Candidate Queue, an AI Curator, a separate Memory Router, deterministic policy checks, versioned writes, scheduled maintenance, exception review, and minimum-context Resume Packs will control information movement. The design is grounded in research on agent memory, experiential learning, hierarchical working memory, repository memory, staleness, and memory benchmarks. Capstone II will compare a semi-manual three-tier baseline with the automated lifecycle while holding the model, repository, tasks, tests, schema, and resource limits constant. Stage One will replay sixty labeled memory events three times. Stage Two will run six connected coding tasks in three paired sequence repetitions per condition. Planned measures include human memory actions, resumption delay, routing and operation accuracy, incorrect durable writes, recall precision and coverage, software-task success, integrity failures, and cost. The expected outcome is a working local prototype, an inspectable event benchmark, and preliminary evidence about when memory automation reduces human effort without weakening memory or software quality.

Table of Contents

AI Use Disclosure **2**

Abstract **3**

List of Figures **8**

List of Tables **8**

List of Abbreviations and Acronyms **9**

Glossary of Key Terms **9**

Chapter 1 Introduction and Problem Definition **10**

1.1 Background and Context . . . 10
1.2 Project Domain . . . 10
1.3 Research or Practice Gap . . . 11
1.4 Problem Statement . . . 11
1.5 Research Question or Project Objective . . . 12
1.6 Supporting Questions or Objectives . . . 12
1.7 Purpose of the Proposed Work . . . 12
1.8 Significance of the Proposed Work . . . 13
1.9 Scope . . . 13
1.10 Delimitations . . . 13
1.11 Assumptions . . . 14
1.12 Constraints . . . 14
1.13 Organization of the Report . . . 14
1.14 Chapter Summary . . . 14

Chapter 2 Literature Review and Foundation **15**

2.1 Introduction . . . 15
2.2 Literature Search Strategy . . . 16
2.2.1 Search Terms and Sources . . . 16
2.2.2 Screening and Evidence Policy . . . 16
2.2.3 Review Limitations . . . 16
2.3 Foundations and Memory Architecture . . . 17
2.3.1 Memory as an External Lifecycle . . . 17

2.3.2 Episodic Memory and Reflection . . . 17
2.3.3 Hierarchy and Structure . . . 18
2.4 Lifecycle and Adaptive Memory Management . . . 18
2.4.1 From Fixed Pipelines to Agent-Controlled Operations . . . 18
2.4.2 Write-Time Trust . . . 19
2.4.3 Task-Aware and Cost-Aware Retrieval . . . 19
2.5 Memory Quality Conflict and Staleness . . . 20
2.5.1 Stale and Conflicting Memory . . . 20
2.5.2 Consolidation and Evidence Loss . . . 20
2.5.3 Static Context and Context Bloat . . . 21
2.6 Repository Memory for Coding Agents . . . 21
2.6.1 Repository History as Memory . . . 21
2.6.2 Trajectory Memory and Durable Project Memory . . . 22
2.6.3 Hierarchical and Shared Repository Memory . . . 22
2.7 Storage and Sharing Scope . . . 22
2.8 Relevant Benchmarks and Evaluation Methods . . . 23
2.8.1 Conversation-Oriented Benchmarks . . . 23
2.8.2 Coding and Agentic Benchmarks . . . 23
2.8.3 Need for a Controlled Comparison . . . 23
2.9 Comparison of Existing Approaches . . . 24
2.9.1 Findings with Stronger Support . . . 24
2.9.2 Unresolved Questions . . . 25
2.10 Limitations and Refined Gap . . . 25
2.10.1 The coding-agent memory lifecycle remains fragmented . . . 25
2.10.2 Write policy is less mature than retrieval . . . 25
2.10.3 Consolidation needs evidence preservation and gates . . . 25
2.10.4 Repository drift is under-evaluated . . . 25
2.10.5 Memory structure and retrieval are task-dependent . . . 25
2.10.6 Static context files are an incomplete baseline . . . 25
2.10.7 Team and cross-agent memory lack governance evaluation . . . 26
2.10.8 Long-horizon coding benchmarks rarely isolate memory . . . 26
2.10.9 Direct evidence is recent and needs replication . . . 26
2.11 Technical Foundation and Design Implications . . . 26
2.12 Chapter Summary . . . 27
2.9.3 Representative Comparison of Existing Approaches . . . 28
2.11.1 Literature Findings and Design Implications . . . 29

Chapter 3 Proposed Solution and Technical Design 30

3.1 Introduction . . . 30
3.2 Solution Overview . . . 30
3.3 Design Goals . . . 30
3.4 Stakeholders and Intended Users . . . 30
3.5 Solution Requirements . . . 31
3.6 Proposed Architecture Workflow or Conceptual Model . . . 31
3.7 Major Components . . . 32
3.8 Data Inputs Outputs and Interfaces . . . 33
3.9 Role of Artificial Intelligence . . . 33
3.10 Human Oversight and Validation . . . 34
3.11 Security Privacy and Ethical Considerations . . . 34
3.12 Traceability to the Problem and Literature . . . 34
3.13 Feasibility . . . 35
3.14 Proposed Capstone II Artifact . . . 35
3.15 Chapter Summary . . . 35

Chapter 4 Evaluation and Research Methodology 36

4.1 Introduction . . . 36
4.2 Evaluation Purpose . . . 36
4.3 Evaluation Questions . . . 36
4.4 Research and Evaluation Design . . . 36
4.5 Units of Analysis Cases and Test Subjects . . . 37
4.6 Data Sources . . . 38
4.7 Variables Metrics and Measures . . . 39
4.8 Baseline or Comparison Method . . . 39
4.9 Data Collection Instruments . . . 40
4.10 Evaluation Procedure . . . 40
4.11 Data Management . . . 40
4.12 Data Analysis Plan . . . 40
4.13 Success Criteria . . . 41
4.14 Reliability and Validity . . . 41
4.15 Threats to Validity . . . 41
4.16 Ethical Privacy and Security Considerations . . . 42
4.17 Limitations . . . 42

4.18 Chapter Summary . . . 42

Chapter 5 Capstone II Execution Plan 43

5.1 Introduction . . . 43
5.2 Work Breakdown . . . 43
5.3 Capstone II Milestones . . . 43
5.4 Schedule . . . 43
5.5 Required Resources . . . 44
5.6 Technical Environment . . . 45
5.7 Dependencies . . . 45
5.8 Risks and Mitigation . . . 45
5.9 Quality Assurance . . . 46
5.10 Configuration and Version Management . . . 46
5.11 Documentation Plan . . . 46
5.12 Completion and Acceptance Criteria . . . 47
5.13 Chapter Summary . . . 47

Chapter 6 Expected Contributions and Conclusion 48

6.1 Proposal Summary . . . 48
6.2 Expected Artifact or Research Outcome . . . 48
6.3 Expected Contribution to Practice . . . 48
6.4 Expected Contribution to Knowledge . . . 48
6.5 Anticipated Limitations . . . 48
6.6 Readiness for Capstone II . . . 49
6.7 Conclusion . . . 49

References 50

Appendix A Literature Synthesis Matrix 52

Appendix B Planned Data Dictionary 55

Appendix C Evaluation Protocol 56

Appendix D Formal Event Families and Coding Sequence 57

List of Figures

Figure 1. Continuity gap between temporary agent sessions

Figure 2. Automated three-tier external-memory architecture

Figure 3. Evidence-aware memory lifecycle

Figure 4. Two-stage evaluation design

Figure 5. Capstone II execution sequence

List of Tables

Table 1. Project definition and alignment

Table 2. Representative comparison of existing approaches

Table 3. Literature findings and design implications

Table 4. Proposed solution requirements

Table 5. Component responsibilities and safeguards

Table 6. Problem-design-evaluation traceability

Table 7. Evaluation questions and evidence

Table 8. Primary experimental conditions

Table 9. Connected coding tasks

Table 10. Metrics and decision criteria

Table 11. Data sources and instruments

Table 12. Capstone II schedule and milestones

Table 13. Required resources

Table 14. Risk register

List of Abbreviations and Acronyms

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
EQ	Evaluation Question
JSON	JavaScript Object Notation
JSONL	JSON Lines
LLM	Large Language Model
RQ	Research Question
SDLC	Software Development Life Cycle

Glossary of Key Terms

Term	Definition in this proposal
Agent memory	External information preserved and later made available to an agent system; it does not refer to model-weight training.
Project Memory	Stable, validated repository knowledge such as commands, constraints, conventions, and confirmed decisions.
Daily Log	Append-only chronological evidence of events, commands, changes, validation, and timestamps.
Session Memory	Goal, progress, tentative findings, failures, blockers, and next steps for one resumable logical task.
Hook	An adapter that observes an approved lifecycle event and creates an evidence-bearing candidate without writing canonical memory.
Curator	The AI-assisted component that interprets an event and proposes claims, uncertainty, and possible conflicts.
Router	The component that selects the destination tier, memory operation, and minimum recall scope.
Policy Gate	Deterministic checks that accept, block, or escalate a proposed memory action.
Cron maintenance	Scheduled invocation of the same guarded lifecycle for revalidation, deduplication, archival, and garbage-collection candidacy.
Resume Pack	A bounded and auditable set of memory records selected for a new coding-agent session.
Memory drift	A condition in which stored memory no longer matches current repository evidence or project decisions.

Chapter 1 Introduction and Problem Definition

1.1 Background and Context

Large language model based coding agents can inspect repositories, edit files, run commands, and respond to validation results. These capabilities allow a software task to continue through several cycles of planning, implementation, testing, and repair. The model invocation that supports one cycle, however, is temporary. A later session does not automatically inherit a reliable record of architectural decisions, failed approaches, current work, or the evidence that justified earlier conclusions. The repository persists, but the reasoning context around the repository can disappear.

Developers compensate by repeating instructions, maintaining project files, preserving chat transcripts, or creating summaries. These practices can help, but they leave the user responsible for deciding what to save, where to save it, when to revise it, and what to reload. Relevant information can therefore become scattered across code, tests, prompts, summaries, and notes. The agent may repeat a failed approach, use an outdated path, overlook a still-valid constraint, or consume a large context window with material unrelated to the current task.

This continuity problem becomes more important in AI software-factory workflows. Such workflows expect an agent to perform multiple software-engineering activities with limited human intervention. Reliable continuation requires more than storing a long transcript. The surrounding system needs explicit rules for creating, validating, routing, updating, retaining, and retrieving external memory. This proposal treats memory as software infrastructure whose behavior should be observable and evaluated.

Project evidence persists, but the model session does not

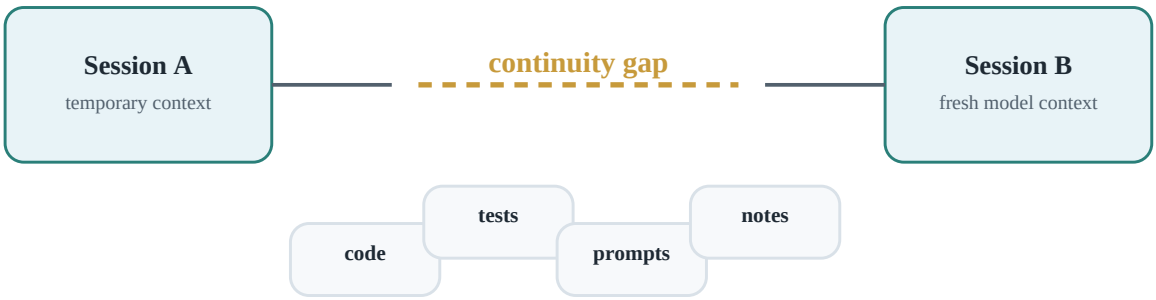


Figure 1. Continuity gap between temporary agent sessions. Source: authors' proposed conceptual model.

1.2 Project Domain

The project belongs to the Techniques area of software engineering in the era of AI, specifically filesystem memory and hierarchical summaries for long-horizon coding agents. Its primary object of study is external memory managed by an agent system. External memory includes durable project facts, chronological evidence, and task-specific state stored outside model parameters and reintroduced when needed.

The proposed work does not modify model weights and does not attempt to reproduce human cognition. It focuses on a bounded engineering question: how a coding-agent environment can manage useful project knowledge across separate sessions while preserving evidence, limiting unsafe writes, and controlling recall cost. StrongDM-style software-factory concepts provide implementation context for event-driven coding workflows, but the academic argument rests on the verified agent-memory and software-engineering literature reviewed in Chapter 2.

1.3 Research or Practice Gap

Research has established several useful memory mechanisms. Agents can store reflections derived from feedback, compress completed subgoals, retrieve semantically related experiences, reuse repository history, and learn memory operations. Coding-specific studies show that repository history and adaptive context management can improve particular tasks. These results demonstrate that external memory can matter.

The literature does not yet provide a mature, auditable lifecycle for coding-agent memory. Existing approaches usually emphasize one part of the problem: retrieval, compression, reflection, repository localization, or experience sharing. They do not consistently connect memory categories, event triggers, evidence and provenance, tier selection, conflict handling, scheduled maintenance, selective forgetting, and resumption across related repository tasks. Recent studies also show that memory can become harmful when outdated evidence remains active, similar experience over-directs later behavior, or repeated model-based consolidation corrupts useful information.

The refined gap is therefore incomplete lifecycle integration and limited coding-specific evaluation. The project does not claim that coding agents have no memory. It addresses the absence of a transparent and empirically testable architecture for moving information among short-lived task state, chronological evidence, and durable project knowledge.

1.4 Problem Statement

Long-horizon coding agents lack a consistent and transparent mechanism for managing durable external memory across otherwise independent model interactions. Project knowledge is distributed across code, validation traces, instructions, summaries, and conversation history, while users remain responsible for many save, update, correction, and recall decisions. This condition can cause lost decisions, repeated failed approaches, stale guidance, inconsistent resumption, unnecessary context growth, and untraceable durable claims.

The proposed work addresses this problem by designing and evaluating an automated three-tier external-memory lifecycle. The design will separate stable project knowledge, chronological evidence, and resumable task state; connect memory changes to source evidence; use AI only for bounded semantic interpretation; apply deterministic controls to durable writes; and retrieve a minimum auditable context for later work.

Table 1. Project Definition and Alignment

Element	Approved statement
Gap	Memory capabilities exist, but lifecycle integration and coding-specific evaluation remain fragmented.
Problem	Users still manage durable context across temporary coding-agent sessions, creating continuity and quality risks.
Objective	Design and evaluate an automated, auditable three-tier external-memory lifecycle.
Primary comparison	Semi-manual and automated control of the same three-tier store.
Expected contribution	Prototype architecture, memory-event benchmark, and preliminary feasibility evidence.

1.5 Research Question or Project Objective

The primary objective is to design and evaluate a practical framework and bounded prototype for automated external-memory management in long-horizon software-engineering tasks.

The main research question is: How can event-driven and scheduled lifecycle automation reduce manual memory-management work for coding agents while preserving memory integrity and software-task performance?

Primary research question: How can event-driven and scheduled lifecycle automation reduce manual memory-management work for coding agents while preserving memory integrity and software-task performance?

1.6 Supporting Questions or Objectives

The project is guided by three supporting questions:

- How should durable project knowledge, chronological evidence, and resumable task state be separated and represented?
- Can automated writing, routing, maintenance, and recall reduce manual effort and resumption delay while maintaining correct writes and relevant recall?
- Across connected coding tasks, does lifecycle automation preserve task success and reduce repeated or contextual errors at an acceptable cost?

1.7 Purpose of the Proposed Work

The purpose of the work is to convert the memory lifecycle from an informal user responsibility into an inspectable software process. Capstone II will produce a local prototype, a controlled set of memory-event cases, a sequence of connected coding tasks, and an evidence-linked comparison between a semi-manual three-tier baseline and an automated version of the same memory foundation.

The project will examine both component correctness and end-to-end usefulness. Component evaluation will ask whether the system captures, interprets, routes, validates, and retrieves memory appropriately.

End-to-end evaluation will ask whether those decisions reduce manual effort and resumption delay without causing unacceptable software-quality or integrity failures.

1.8 Significance of the Proposed Work

The proposed framework may help developers and small software teams reason about memory as a managed engineering artifact rather than an opaque product feature. A transparent architecture can show why a memory was created, which evidence supports it, which tier owns it, and when it was last verified. These properties are relevant to researchers who need reproducible experiments, instructors who teach agentic software engineering, and developers who use coding agents across multiple tasks.

The study also has methodological value. It will hold the memory schema constant while varying lifecycle automation, and it will connect each observed coding outcome to the underlying event, decision, memory version, and recall package. This design can reveal useful trade-offs even if the automated condition does not meet every success criterion.

1.9 Scope

The implementation scope is one local Python repository, one pinned coding-agent and model configuration, three human-readable memory tiers, focused lifecycle hooks, a lightweight candidate queue, an AI Curator, a separate Router, deterministic policy checks, scheduled maintenance invoked through a controlled clock, and a selective Resume Pack Builder. The evaluation will use synthetic or controlled inputs, sixty formal memory-event cases, six connected coding tasks, and three paired sequence repetitions.

The research scope includes external agent memory, lifecycle events, provenance, routing, conflict escalation, versioned writes, retention and archival decisions, minimum-context recall, manual effort, resumption, software tests, and operational cost. It is designed as a feasibility study rather than a claim about production-scale reliability.

1.10 Delimitations

The project will not train or fine-tune a foundation model, alter model parameters, implement enterprise identity or multi-tenancy, deploy to a production codebase, or claim universal cross-agent memory. It will not evaluate every coding model, language, repository type, or memory architecture. Team-wide governance, distributed concurrent writes, large-scale organizational privacy controls, and long-term production retention remain outside the prototype boundary.

The primary comparison is semi-manual versus automated operation of the same three-tier store. A no-memory or static-summary baseline may be explored only after the primary study is complete; it is not required for the main decision.

1.11 Assumptions

Planning assumes that a pinned model interface, local development tools, Git, Python, pytest, and a scheduler remain available during Capstone II. It assumes that controlled repository fixtures can expose memory-relevant events without requiring confidential code. It also assumes that model output will vary across executions and must therefore be evaluated through repeated runs and preserved traces rather than treated as deterministic truth.

The design assumes that source evidence can be represented through file references, commit identifiers, test results, timestamps, or content hashes. A passed test supports the behavior it checks; it does not validate every natural-language claim produced by an agent.

1.12 Constraints

The study is constrained by one academic term, limited model budget, a small repository, a small number of paired sequences, and the dual role of the researchers as designers, operators, and annotators. Some directly relevant coding-memory studies are recent preprints, and the field lacks settled measurement standards. These constraints limit generalization and place a premium on transparent procedures, frozen rubrics, independent annotation, and conservative conclusions.

1.13 Organization of the Report

Chapter 2 synthesizes research on agent memory, adaptive lifecycle management, memory quality, repository memory, and benchmarks. Chapter 3 translates that evidence into the three-tier architecture and automated control plane. Chapter 4 defines the controlled evaluation, data sources, metrics, procedures, and validity protections. Chapter 5 provides the Capstone II work plan, resources, risks, and completion criteria. Chapter 6 summarizes the expected artifact, anticipated contributions, limitations, and readiness to proceed.

1.14 Chapter Summary

Coding-agent sessions are temporary while software projects persist. Existing memory mechanisms demonstrate useful capabilities but do not provide a complete, auditable lifecycle for coding work. This project proposes a bounded three-tier architecture and tests whether lifecycle automation reduces routine human work without weakening memory integrity or coding-task performance. The narrow prototype, controlled comparison, and explicit boundaries make the project suitable for Capstone II.

Chapter 2 Literature Review and Foundation

2.1 Introduction

LLM-based coding agents are compound software systems. The language model provides reasoning and generation, while the surrounding agent supplies tools, repository access, command execution, validation, context construction, and control flow. Systems such as SWE-agent demonstrate that the design of the agent-computer interface materially affects repository navigation, editing, and testing performance (Yang et al., 2024). CodePlan similarly shows that repository-wide change requires explicit planning over interdependent files and iterative feedback, not merely local code completion (Bairi et al., 2024). These systems illustrate why long-horizon software engineering creates a memory problem: the information needed to act correctly is distributed across source code, project history, earlier tool results, architectural decisions, validation outcomes, and prior failed attempts.

Agent memory should not be equated with everything that appears in a model prompt. The peer-reviewed survey by Zhang et al. (2025) describes memory as a module that participates in the agent-environment loop: information is collected, processed into usable forms, and later used to guide action. A recent preprint survey further distinguishes agent memory from generic retrieval- augmented generation and context engineering by emphasizing how memory is formed, evolves, and is retrieved over time (Hu et al., 2025). The distinction matters for software engineering. Searching the current repository can retrieve code, but it may not explain why an earlier design was rejected, which tests validated a behavior, or which project convention emerged through years of commits.

This review uses four questions developed from the project team's planning notes:

1. **What should be stored?** Useful candidates include repository facts, architecture, standards, rules, plans, episodes, failed attempts, validation history, and project-specific skills. 2. **When should memory be managed?** A system needs triggers for writing, updating, retrieving, consolidating, deprecating, or forgetting memory. 3. **How should memory quality be managed?** Duplicate, conflicting, uncertain, and stale information can make later actions worse. 4. **Where should memory live?** An agent-local store, checked-in repository guidance, team-shared memory, and cross-agent memory have different governance and privacy implications.

The scope is external memory managed by the agent system. Model-weight editing, fine-tuning as memory, human cognitive simulation, and hardware or KV-cache memory are outside the central analysis. Conversation-memory studies are included only when their mechanisms or benchmarks clarify a problem that also applies to coding agents.

2.2 Literature Search Strategy

2.2.1 Search Terms and Sources

The review used a focused narrative method with a documented protocol rather than claiming an exhaustive systematic review. Searches were conducted on July 26, 2026 across the ACM Digital Library, ACL Anthology, AAAI Digital Library, ICLR Proceedings and OpenReview, NeurIPS Proceedings, Proceedings of the ACM on Software Engineering, arXiv, Microsoft Research publication pages, and Google-Scholar-style web discovery. Search families combined terms for LLM agent memory, dynamic and adaptive memory management, coding-agent memory, repository memory, hierarchical and episodic memory, conflict and staleness, shared memory, and long-horizon evaluation.

Examples included `"repository memory" AND "coding agents"`, `"adaptive memory management" AND "LLM agents"`, `"memory conflict resolution" AND agents`, `"long-horizon coding agents" AND memory`, and `"memory benchmark" AND coding`. Exact strings, source locations, and criteria are recorded in [`01_search_protocol.md`](01_search_protocol.md).

2.2.2 Screening and Evidence Policy

Thirty-five academic candidates were retained in the source catalog. Twenty-one were designated core sources because they directly proposed, evaluated, or systematically reviewed an external memory mechanism relevant to the four questions. Peer-reviewed journal and conference papers were treated as primary evidence. Preprints were included when they addressed a recent coding-specific mechanism or failure mode not yet represented in mature publications, but their status is stated explicitly.

Official documentation from Anthropic, OpenAI, and StrongDM was reviewed only to describe implementation context. It was not used as a substitute for academic evidence of effectiveness. Marketing pages, unsourced summaries, hardware-memory papers, generic RAG work, and work concerned only with parametric memorization were excluded from the core evidence.

2.2.3 Review Limitations

The field is changing rapidly. Several sources that most directly concern coding-agent memory appeared in 2026 and remain preprints. Search-engine ranking is not stable, and terminology is inconsistent across papers. Reported benchmark results also cannot be compared as a single leaderboard because systems differ in model, tool interface, task, context budget, and evaluation method. The review therefore emphasizes mechanisms, converging observations, and evidence maturity rather than declaring one universal architecture.

2.3 Foundations and Memory Architecture

2.3.1 Memory as an External Lifecycle

The most useful common definition is operational: external agent memory is information persisted beyond the immediate model call and later selected or transformed to influence action. Zhang et al. (2025) organize prior work by memory source, form, and operation, emphasizing that design and evaluation are both necessary. The more recent taxonomy of Hu et al. (2025) describes factual, experiential, and working memory and analyzes formation, evolution, and retrieval. Together, these perspectives show why the familiar short-term versus long-term distinction is insufficient. Two memories can both persist for months but serve different functions: one may be a validated architectural fact, while another records a failed debugging episode.

For coding agents, at least five functional categories recur in the literature:

- **Working memory:** the current plan, active subgoal, recent observations, and tool outputs needed for the present task.
- **Episodic memory:** what the agent attempted, what happened, and what feedback followed.
- **Semantic repository memory:** relatively stable facts about modules, dependencies, APIs, and current functionality.
- **Procedural or skill memory:** reusable methods, such as how this repository adds an endpoint or validates a migration.
- **Normative memory:** instructions, constraints, and standards that define acceptable behavior.

These categories are analytically distinct even when all are stored as text. The representation alone does not determine function. A Markdown entry can be a rule, an episode, a summary, or an uncertain hypothesis. Consequently, a memory schema must identify purpose and evidence, not merely file location.

2.3.2 Episodic Memory and Reflection

Generative Agents is a canonical architecture combining an observation stream, retrieval, reflection, and planning (Park et al., 2023). It stores experiences in natural language, ranks memories by recency, importance, and relevance, and synthesizes higher-level reflections when accumulated importance reaches a threshold. Its social-simulation setting is distant from coding, but the architecture establishes an influential pattern: raw episodes and consolidated abstractions coexist.

Reflexion brings this pattern closer to software work. Rather than updating model weights, it converts task feedback into verbal reflections stored in an episodic buffer and reuses those reflections on later trials (Shinn et al., 2023). The evaluation includes coding, sequential decision-making, and reasoning tasks. The study provides evidence for a central project intuition: a failed attempt or test result can become useful

memory if it is transformed into actionable feedback.

ExpeL extends experience reuse across tasks. It collects trajectories, extracts natural-language insights, and retrieves both examples and distilled lessons at inference time (Zhao et al., 2024). Experiential Co-Learning applies related ideas to software-developing agents, where instructor and assistant agents extract shortcut-oriented knowledge from historical trajectories and reuse it on unseen tasks (Qian et al., 2024). These studies support external experiential learning without parameter updates. They do not, however, guarantee that the extracted lesson is true, current, or appropriate for a different repository state.

2.3.3 Hierarchy and Structure

MemGPT treats limited context as a virtual-memory problem and gives the agent operations for moving information between model-visible and external tiers (Packer et al., 2023). Although the paper is a preprint, the operating-system analogy made an important architectural point: memory is not only a database; it is a control problem involving limited fast context and larger slower stores.

HiAgent provides stronger peer-reviewed evidence for hierarchy within a long-horizon task. It groups action-observation history by subgoal and allows the agent to replace completed subgoal traces with summaries while retaining the evidence relevant to the active subgoal (Hu et al., 2025). Across five long-horizon tasks, the authors report a twofold increase in success rate and 3.8 fewer steps on average. The result supports task-aware compression rather than a single rolling summary.

A-MEM represents memory as interconnected structured notes with contextual descriptions, keywords, tags, embeddings, and links. Adding a new memory can alter the attributes or relationships of prior memories (Xu et al., 2025). FluxMem goes further by arguing that no single structure fits every interaction; it learns to select among multiple structures and reports gains on long-horizon conversation benchmarks (Lu et al., 2026). Both are preprints, but they reveal a useful research direction: choosing the memory structure may itself need to be adaptive.

For repositories, structural memory is especially important because code has directory, dependency, type, call, and test relationships. LocAgent demonstrates that a heterogeneous code graph supports strong multi-hop localization (Chen et al., 2025). This is not durable memory across tasks, but it establishes that semantic similarity alone is an incomplete representation of repository knowledge.

2.4 Lifecycle and Adaptive Memory Management

2.4.1 From Fixed Pipelines to Agent-Controlled Operations

Early systems generally hard-code the memory lifecycle. Observations are saved, reflections occur after a threshold, or a summary is generated after a fixed amount of history. Such rules can be useful, but they assume that the same event schedule and granularity fit every task.

Recent peer-reviewed work treats memory operations as part of the agent policy. Agentic Memory (AgeMem) exposes store, retrieve, update, summarize, and discard as tools and learns a unified policy over short- and long-term memory using reinforcement learning (Yu et al., 2026). Across five long-horizon benchmarks, the framework outperforms strong memory-augmented baselines while improving context

efficiency. The significance is conceptual as well as empirical: "remember" is no longer a fixed preprocessing step; it is an action selected in relation to task progress.

Related recent work pursues the same transition. Memory-R1 learns ADD, UPDATE, DELETE, and NOOP decisions using a memory manager and an answer agent (Yan et al., 2025). Memory-as-Action places working-memory edits inside the main policy, allowing the agent to curate its own context during long tasks (Y. Zhang et al., 2026). SWE-MeM applies proactive and on-demand memory management directly to software-engineering agents. It allows the agent to decide when, what, and how to compress according to trajectory state, progress, and remaining context budget (Gao et al., 2026). On SWE-bench Verified, the authors report resolve rates of 43.4% and 60.2% for 4B and 30B models respectively, with improved efficiency relative to static compression baselines.

These results support dynamic management, but they do not imply that a capstone must train a reinforcement-learning policy. Learned policies are expensive, difficult to reproduce, and optimized for benchmark reward rather than necessarily for memory integrity. A transparent rule-based lifecycle can still be dynamic if it responds to meaningful events: successful validation, a changed interface, a repeated failure, an explicit user correction, or completion of a project phase.

2.4.2 Write-Time Trust

Retrieval receives more attention than durable write quality. Reflexion and ExpeL distill experience, A-MEM evolves notes, and learned systems select write operations, but an agent can transform a mistaken interpretation into a confident durable lesson. Xiong et al. (2026) show that memory addition and deletion materially change long-term behavior. They identify "experience-following": when a current input resembles a retrieved memory, the agent often produces a correspondingly similar output. This makes a bad memory more than an archival error; it becomes a behavioral control input.

For software engineering, an evidence-grounded write policy is therefore essential. A repository fact can be linked to a file and commit; a behavioral claim can be linked to a test or scenario; a decision can be marked user-approved; an unverified inference can remain tentative. The reviewed literature demonstrates pieces of this design, but not a broadly adopted coding- agent standard.

2.4.3 Task-Aware and Cost-Aware Retrieval

Selective retrieval is necessary because loading everything recreates the context-window problem. Generative Agents combines relevance, recency, and importance. ExpeL retrieves similar experiences and general lessons. HiAgent keeps active-subgoal evidence. Repository methods add structural and historical signals. These approaches imply that coding memory retrieval should combine:

- current task and subgoal relevance;
- repository location and dependency relationships;
- memory type;
- recency and version;
- validation status and confidence;

- cost or context budget.

This is more than top-k vector similarity. A failed test tied to the same module may matter even if its natural-language description is not semantically close to the current request. Conversely, a highly similar memory may be obsolete.

2.5 Memory Quality Conflict and Staleness

2.5.1 Stale and Conflicting Memory

Memory systems face a basic temporal problem: information changes. STALE evaluates implicit conflicts in which later evidence invalidates an earlier belief without explicitly stating the negation (Chao et al., 2026). Across 400 expert-validated scenarios and 1,200 queries, the best evaluated model achieved only 55.2% overall accuracy. The study reports a gap between retrieving updated evidence and behaving consistently with it. A model may find the new information yet still accept an outdated premise in the user's question.

This failure maps directly to repositories. A module move, framework migration, or changed validation requirement may invalidate several related memories even when no commit message says "the prior memory is now false." Repository drift therefore involves inference over dependencies and versions, not only replacing one string with another.

Reddy and Challaram (2026) offer a narrower corrective. On explicit versioned facts, a deterministic aggregation rule can outperform LLM-mediated freshness resolution. Their results suggest that current-value selection with clear serials or timestamps belongs in conventional software logic. The authors also show that this solution does not automatically generalize to every knowledge- update question. Deterministic rules and LLM reasoning must be composed rather than treated as alternatives for all cases.

2.5.2 Consolidation and Evidence Loss

Continuous consolidation is often presented as a path to self-improving memory. However, D. Zhang et al. (2026) report that repeatedly asking LLMs to rewrite past experience into a consolidated memory can cause utility to rise and then degrade, sometimes below a no-memory baseline. Their controlled analysis traces the failure to consolidation rather than to the underlying episodes. Agents that retained raw episodes remained competitive with or better than forced consolidators.

This result challenges a naive event rule such as "after every task, rewrite the repository memory." A safer lifecycle preserves raw evidence and treats consolidation as a gated, reversible transformation. For a coding agent, the trace, diff, command result, and test output should remain available even when a short lesson is generated. The lesson should point to evidence rather than replace it.

2.5.3 Static Context and Context Bloat

Repository instruction files are a practical form of durable external context, but they illustrate the cost of uncontrolled accumulation. Gloaguen et al. (2026) evaluate coding agents with generated and developer-authored `AGENTS.md`-style context files. They report that context files tended to lower task success and increase inference cost by more than 20%, even though agents often followed the instructions and explored more broadly. The authors conclude that unnecessary requirements make tasks harder and recommend minimal files.

dos Santos et al. (2026) mine 100 repositories and identify configuration smells including context bloat, conflicting instructions, and misplaced procedural detail. Their study is a preprint and its heuristic smells do not independently prove causal performance effects, but it provides maintainability evidence. Together, these works reject the assumption that a large `AGENTS.md`, `CLAUDE.md`, or flat memory file is a sufficient long-term-memory strategy.

2.6 Repository Memory for Coding Agents

2.6.1 Repository History as Memory

The strongest direct peer-reviewed evidence is *Improving Code Localization with Repository Memory* (Wang et al., 2026). Existing localization agents generally begin each issue without accumulated repository knowledge. Wang and colleagues construct non-parametric memory from recent commits and linked issues, together with functionality summaries for actively evolving parts of the codebase. Tools allow LocAgent to retrieve this memory while localizing code. The method improves localization on SWE-bench Verified and SWE-bench Live.

The study demonstrates that repository history contains useful knowledge beyond the current snapshot. A source tree shows what exists now; commits and issues can reveal associations between bug types, modules, and prior change patterns. However, localization is only one stage. The study does not evaluate an integrated memory lifecycle across implementation, validation, repair, and later tasks.

Learning to Commit extends historical memory to patch quality. It uses a strict chronological split: an agent attempts earlier historical issues, compares its attempt with the oracle diff, and distills repository-specific skills concerning style, internal API use, and architectural invariants (Li et al., 2026). Future pull requests are conditioned on the resulting online repository memory. The reported evaluation includes functional correctness, style, internal API reuse, and modified-region plausibility. This is highly relevant to the team's observation that some memory is indirectly related to the immediate task but strongly related to the project's output, such as coding style.

The evidence is promising but preliminary. The paper is a preprint and reports an expert-maintained repository rather than broad multi-repository evaluation. Its skill construction also uses oracle historical diffs, which a live system does not have for new tasks.

2.6.2 Trajectory Memory and Durable Project Memory

SWE-MeM manages the current coding trajectory under a context budget (Gao et al., 2026). This is valuable working memory, but it is not identical to durable repository knowledge. A compressed trajectory may help finish one issue while providing little reusable information for the next session. Conversely, a stable architecture decision may not be needed in the current working context but should remain available for future tasks.

This distinction suggests two linked layers:

1. **trajectory memory**, optimized for completing the active task; and 2. **repository memory**, optimized for preserving validated knowledge across tasks and sessions.

An integrated lifecycle needs a promotion rule from the first layer to the second. A test failure should not automatically become a permanent rule. A repeated, validated pattern may deserve promotion with supporting evidence.

2.6.3 Hierarchical and Shared Repository Memory

MemDocAgent uses dependency-aware traversal and a shared RepoMemory with read, write, and verify operations to generate consistent hierarchical repository documentation (Bae et al., 2026). The task is documentation rather than code change, but it directly demonstrates how prior work traces can support later repository components. It also aligns with pyramid memory: file-level work contributes to module- and repository-level representations.

Experiential Co-Learning offers peer-reviewed evidence that software-developing agents can share extracted experience and improve performance on unseen tasks (Qian et al., 2024). Nevertheless, neither study deeply evaluates team governance. If several agents or developers can write shared memory, a usable system needs provenance, permissions, review, conflict handling, rollback, and possibly different visibility scopes.

2.7 Storage and Sharing Scope

The literature provides mechanisms for personal, agent-specific, and shared memory, but there is no clear consensus on storage scope. Gao and Zhang (2024) propose a shared memory pool in which multiple agents contribute filtered prompt-answer pairs and a retriever improves over time. The reported open-ended task results suggest that shared experience can help, but the setting says little about repositories, private code, conflicting updates, or team authority.

For software engineering, storage location is a governance decision:

- **agent-local memory** can contain personal workflow experience but is not automatically visible or reviewable by teammates;
- **repository-checked-in memory** is auditable through version control but can create context bloat and requires maintainers;
- **team-shared services** can support cross-machine and cross-agent access but

require permissions, privacy controls, and concurrent update semantics;

- **task-local traces** provide detailed evidence but may be too noisy or sensitive for durable sharing.

Current products illustrate this separation. Anthropic documents project instructions in `CLAUDE.md` alongside machine-local auto memory containing a `MEMORY.md` index and topic files. OpenAI documents required repository guidance in `AGENTS.md` separately from experimental local memories generated from eligible prior chats. These are useful examples of current implementation choices, not academic proof that the choices are optimal. StrongDM's Attractor specification describes an observable coding loop with tools, context management, and stop conditions; it provides a natural host architecture in which a memory layer could be studied, but it is not itself an evaluation of repository memory.

The gap is therefore not that storage options are unknown. The gap is that the software-engineering consequences of scope, write authority, sharing, and rollback remain weakly evaluated.

2.8 Relevant Benchmarks and Evaluation Methods

2.8.1 Conversation-Oriented Benchmarks

LoCoMo evaluates question answering, event summarization, and generation over very long multi-session conversations (Maharana et al., 2024). LongMemEval tests information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention (Wu et al., 2025). These benchmarks made long-term update and temporal reasoning measurable. They remain oriented toward assistant histories rather than memory generated during active software work.

MemoryAgentBench broadens the competency model. It evaluates accurate retrieval, test-time learning, long-range understanding, and selective forgetting through incremental multi-turn interactions (Hu et al., 2026). Its results show that current systems do not master all four competencies. This is important for the team's project because a memory system should not be judged only by whether it can retrieve a saved fact.

2.8.2 Coding and Agentic Benchmarks

SWE-bench provides executable, test-based evaluation of real GitHub issues (Jimenez et al., 2024). It is a natural base for coding-agent research, but its standard formulation primarily treats issues independently. It therefore does not isolate whether an agent accumulates useful repository memory across tasks.

MemGym explicitly identifies this mismatch. The preprint unifies coding, tool use, computer use, and deep research behind a memory-reasoning interface and reports memory-isolated scores intended to separate memory effects from reasoning and tool-use ability (Xu et al., 2026). It is among the most directly relevant evaluation proposals, but it is new and some coding evaluation relies on a learned reward-model proxy instead of full repository rollouts.

2.8.3 Need for a Controlled Comparison

Memory effects are confounded with:

- base model capability;

- repository and task difficulty;
- retrieval quality;
- agent-computer interface;
- context and token budget;
- number of attempts;
- validation quality.

A capstone experiment should hold these variables as constant as practical and vary the memory condition. More importantly, it should use a sequence of related tasks in the same repository. Appropriate outcomes include:

- task and scenario success;
- test pass rate;
- repair iterations;
- repeated failed approaches;
- context or architecture errors;
- regressions;
- token usage and elapsed time;
- stale or conflicting memories retrieved;
- agreement between memory claims and repository evidence.

This design evaluates memory as a behavioral component of the coding system, not as a standalone text-retrieval service.

2.9 Comparison of Existing Approaches

2.9.1 Findings with Stronger Support

First, external experience can improve later agent behavior without updating model weights. Reflexion, ExpeL, and Experiential Co-Learning support this conclusion across different tasks, including coding and software-development settings (Shinn et al., 2023; Zhao et al., 2024; Qian et al., 2024).

Second, hierarchy and selective context management can outperform retaining a long undifferentiated history. HiAgent provides peer-reviewed evidence, while MemGPT and A-MEM contribute influential system designs (Hu et al., 2025; Packer et al., 2023; Xu et al., 2025).

Third, memory operations can be made explicit and adaptive. AgeMem provides the strongest peer-reviewed example of learned store, retrieve, update, summarize, and discard behavior; several recent preprints extend the idea to working context and coding trajectories (Yu et al., 2026; Y. Zhang et al., 2026; Gao et al., 2026).

Fourth, repository history can improve coding-agent localization, and shared experience can improve software-development agents (Wang et al., 2026; Qian et al., 2024). Repository memory is therefore an

empirical research area rather than only a product concept.

2.9.2 Unresolved Questions

The literature does not establish a universal memory schema for coding agents. Different studies benefit from episodes, reflections, graphs, summaries, commits, or skills. Adaptive-structure work suggests that one representation may not fit all interactions (Lu et al., 2026).

It also does not establish that automatic memory growth is safe. Experience following, consolidation degradation, stale-memory failures, and context-file cost all show ways in which memory can lower performance (Xiong et al., 2026; D. Zhang et al., 2026; Chao et al., 2026; Gloaguen et al., 2026).

Finally, the field does not yet provide mature evidence for multi-user and cross-agent repository memory governance. Shared-memory studies focus on task performance, not permissions, provenance, concurrent updates, rollback, or privacy.

2.10 Limitations and Refined Gap

2.10.1 The coding-agent memory lifecycle remains fragmented

Research separately addresses working-memory compression, episodic reflection, repository history, static instructions, shared experience, and conflict resolution. No mature coding-agent framework connects memory type, formation, validation, retrieval, evolution, and retirement in one auditable lifecycle.

2.10.2 Write policy is less mature than retrieval

Many systems retrieve from stored information, while fewer determine whether an observation is trustworthy enough to persist. Coding agents need evidence- grounded writes based on files, diffs, tests, user decisions, and validation results.

2.10.3 Consolidation needs evidence preservation and gates

Recent evidence shows that repeated LLM rewriting can corrupt useful memory. Repository memory should preserve raw traces and sources, make consolidation reversible, and distinguish tentative abstractions from validated facts.

2.10.4 Repository drift is under-evaluated

Memory conflict benchmarks largely focus on personal facts or explicit versions. They do not capture a changing codebase where renamed modules, migrations, dependency changes, and new tests implicitly invalidate multiple memories.

2.10.5 Memory structure and retrieval are task-dependent

Flat text, linked notes, graphs, hierarchical summaries, commits, and skills each support different forms of reasoning. Coding-specific policies for selecting or combining structures remain immature.

2.10.6 Static context files are an incomplete baseline

`AGENTS.md`- or `CLAUDE.md`-style guidance can encode durable rules, but recent evidence shows that unnecessary requirements and context bloat can increase cost and reduce success. Dynamic memory should be compared against a deliberately minimal static baseline.

2.10.7 Team and cross-agent memory lack governance evaluation

Shared experience can improve agent performance, yet coding-specific research rarely evaluates who may write, how claims are reviewed, how conflicts are resolved, and how sensitive or incorrect memories are rolled back.

2.10.8 Long-horizon coding benchmarks rarely isolate memory

SWE-bench evaluates coding outcomes, while many memory benchmarks evaluate assistant recall. The field needs controlled sequences of related repository tasks with fixed model, tools, and budget so that memory contribution can be measured.

2.10.9 Direct evidence is recent and needs replication

Several of the most relevant systems-SWE-MeM, MemDocAgent, Learning to Commit, and consolidation or staleness studies-are 2026 preprints. Their mechanisms are valuable hypotheses, but generalization and independent replication remain open.

2.11 Technical Foundation and Design Implications

The review supports a project narrower than "AI memory" and broader than a single static repository file:

> **Design and evaluate an auditable dynamic external memory-management > framework for long-horizon coding agents.**

A feasible prototype can use a rule-based lifecycle instead of training a new policy. It can separate:

- validated repository facts and architecture;
- episodic attempts and failures;
- validation evidence;
- project rules and standards;
- current task state;
- tentative memories awaiting verification.

Lifecycle events can include successful tests, repeated failures, significant code changes, explicit user corrections, and completion of a development phase. Each durable memory can include a source, repository version, timestamp, confidence or status, and last verification event. Basic drift checks can detect missing paths, changed commands, invalidated tests, or conflicts with newer evidence.

The evaluation should compare:

1. **No persistent memory** 2. **Minimal static instructions or one flat summary** 3. **Structured dynamic memory with evidence-preserving updates**

The prototype would not claim human-like memory, universal cross-agent memory, or superiority across every model and repository. Its contribution would be a transparent lifecycle design and preliminary controlled evidence concerning continuity, repeated mistakes, repair effort, regressions, context cost, and memory consistency.

2.12 Chapter Summary

Agent memory has progressed from simple stored history toward structured, hierarchical, reflective, and policy-managed systems. Peer-reviewed studies show that agents can learn from externalized experience, that hierarchical working memory improves long-horizon execution, that memory operations can be learned, and that repository history can improve code localization. At the same time, the evidence rejects a simplistic "more memory is better" assumption. Bad experience can steer future behavior, continuous consolidation can corrupt useful knowledge, stale information can survive retrieval, and large static context files can increase cost and reduce task success.

For long-horizon coding agents, the central problem is therefore not merely persistence. It is lifecycle governance: what deserves storage, what event should trigger change, what evidence supports a memory, how contradictions and drift are handled, and which scope should receive the result. Current research provides important components but not a mature integrated answer. This makes an auditable, validation-grounded dynamic repository-memory framework a defensible and feasible capstone direction, provided that the prototype and evaluation are kept narrow.

2.9.3 Representative Comparison of Existing Approaches

Table 2. Representative Comparison of Existing Approaches

Approach	Method	Strength	Limitation and relevance
Shinn et al. (2023)	Reflexion converts external or simulated feedback into verbal reflections stored in an episodic buffer and evaluates later trials on decision-making, reasoning, and HumanEval coding tasks, including ablations.	Textual reflection improved later behavior without weight updates; the paper reports 91 percent pass@1 on HumanEval.	A generated reflection can be incorrect, and HumanEval does not represent continuing work in one evolving repository. It directly supports storing failed attempts and validation feedback as external episodic memory.
Packer et al. (2023)	MemGPT implements operating-system-inspired virtual context management with model-visible memory, external archival memory, recall storage, and explicit function calls for moving information.	The prototype demonstrates that an agent can manage information beyond its immediate context in document analysis and multi-session conversation settings.	As a preprint and broad architecture demonstration, it offers limited coding-specific evidence and no repository write-trust rule. It supplies the hierarchy and paging analogy behind separating active trajectory memory from larger durable stores.
Hu et al. (2025)	HiAgent groups working memory by subgoal and allows completed subgoal traces to be replaced with summaries while current evidence remains detailed. It is tested on five long-horizon tasks.	The authors report a twofold success-rate increase and 3.8 fewer steps on average, with improvements across different task lengths.	The system manages in-trial working memory rather than durable, cross-session repository knowledge. It provides strong evidence for task-aware hierarchy and helps us separate trajectory compression from repository memory.
Yu et al. (2026)	AgeMem exposes store, retrieve, update, summarize, and discard as agent tools and learns a unified policy through three-stage reinforcement learning with step-wise GRPO.	Across five long-horizon benchmarks and multiple model backbones, the learned policy improved task performance, memory quality, and context efficiency over memory-augmented baselines.	The training pipeline is expensive and optimizes benchmark reward, not necessarily repository truth, auditability, or team governance. It confirms that memory timing can be part of agent policy while supporting a simpler, auditable rule-based Capstone prototype.
Chao et al. (2026)	STALE constructs 400 expert-validated scenarios and 1,200 queries covering state resolution, premise resistance, and implicit policy adaptation, then evaluates frontier models and memory systems.	The best evaluated system reached only 55.2 percent overall; retrieving updated evidence did not guarantee behavior consistent with the update.	The scenarios concern personal and everyday states rather than repositories, code dependencies, or executable validation. It supplies direct evidence for a memory-drift problem and a model for testing invalidation beyond exact string replacement.
Wang et al. (2026)	The system mines recent commits, linked issues, and functionality summaries into non-parametric memory, adds retrieval tools to LocAgent, and evaluates on SWE-bench Verified and SWE-bench Live.	Repository memory significantly improves the strong localization framework, showing that history contributes information beyond the current code snapshot.	The evaluation concerns localization, not a complete build-test-repair lifecycle, cross-task updates, or team-shared governance. It is the strongest direct academic evidence for repository memory and anchors our coding-specific research gap.
Gao et al. (2026)	SWE-MeM provides a memory tool for deciding when, what, and how to compress, then trains agents with synthesized trajectories and memory-aware GRPO on SWE-bench Verified.	The paper reports 43.4 and 60.2 percent resolve rates for 4B and 30B models, respectively, while improving efficiency over static memory-management baselines.	It is a recent preprint, jointly optimizes issue resolution and compression, and focuses on current-task trajectory memory. It directly validates adaptive management in coding but leaves durable, evidence-grounded repository memory unresolved.
Bae et al. (2026)	MemDocAgent traverses dependencies and uses a shared RepoMemory with read, write, and verify operations so later components can reuse prior traces.	The reported evaluation shows stronger hierarchical consistency and repository-level documentation quality than baselines.	The task is documentation rather than code modification, and concurrent writes, permissions, and provenance are not deeply tested. It demonstrates pyramid-style shared repository memory and motivates separate file, module, and project representations.

2.11.1 Literature Findings and Design Implications

Table 3. Literature Findings and Design Implications

Literature finding	Design implication
External experience can improve later behavior without updating model weights.	Preserve failed attempts, validation outcomes, and confirmed decisions as external records.
Hierarchical and task-aware context can outperform undifferentiated histories.	Separate project, chronological, and session responsibilities and retrieve by task.
Adaptive memory actions are useful but can introduce non-deterministic risk.	Use AI for semantic interpretation while keeping policy and destructive operations bounded.
Stale or repeatedly consolidated memory can reduce performance.	Keep provenance, version links, revalidation, and recoverable archival.
Repository history can support coding tasks.	Ground durable facts in repository evidence and validation results.
Existing benchmarks often fail to isolate memory.	Hold the coding agent and storage schema constant while varying lifecycle automation.

Chapter 3 Proposed Solution and Technical Design

3.1 Introduction

This chapter converts the research problem and literature findings into a proposed technical design. The design separates storage responsibilities, semantic interpretation, routing, deterministic validation, scheduled maintenance, and selective recall. This separation is intended to reduce routine user intervention without allowing an unconstrained model to control durable project knowledge.

3.2 Solution Overview

The proposed artifact is an automated three-tier external-memory manager for long-horizon coding agents. It will capture memory-relevant development events, place them in an evidence-bearing queue, use a bounded AI component to interpret semantic content, route candidate information to an appropriate tier and operation, validate proposed changes through deterministic policy, maintain memory at scheduled checkpoints, and prepare a small auditable context package when a task resumes.

The system is intended for developers, student software teams, and researchers who need continuity across otherwise independent coding-agent sessions. Its inputs include task events, file and commit references, command and test results, user corrections, existing memory state, and a request to resume a task. Its outputs include event traces, candidate claims, routing and policy decisions, versioned memory records, maintenance reports, review items, and Resume Packs.

3.3 Design Goals

The design prioritizes traceability, evidence preservation, controlled automation, selective recall, recoverability, and feasibility. Traceability requires each durable claim to connect to its source. Evidence preservation keeps raw chronological events separate from later summaries. Controlled automation assigns semantic judgment to AI but retains deterministic checks for schema, provenance, sensitive content, version conflicts, and destructive actions. Selective recall limits context to information relevant to the current task. Recoverability requires versioned writes and archival rather than irreversible deletion. Feasibility limits the prototype to local, inspectable components that can be completed and evaluated within one academic term.

3.4 Stakeholders and Intended Users

Primary users are developers who employ coding agents across several related tasks. The system also supports researchers who need inspectable memory experiments and instructors who teach agentic software-engineering methods. The coding agent consumes Resume Packs and produces development activity; it is not treated as the owner of durable memory. The project team operates the prototype, creates controlled fixtures, reviews exceptions, and evaluates outputs. Future organizational administrators and cross-agent services are recognized stakeholders but remain outside the implemented scope.

3.5 Solution Requirements

The requirements below translate the problem into behaviors and quality constraints that can be inspected during Capstone II. Functional requirements define the lifecycle. Quality and safety requirements ensure that reduced human effort does not come from bypassing evidence, review, or recovery.

Table 4. Proposed Solution Requirements

ID	Requirement	Type	Verification
FR1	Capture defined task, validation, correction, and session-boundary events without granting Hooks direct write authority.	Functional	Event-case replay and queue trace
FR2	Maintain separate Project Memory, Daily Log, and Session Memory stores with explicit schemas and version metadata.	Functional	Schema and state inspection
FR3	Extract candidate facts, progress, failures, decisions, uncertainty, and conflict indicators from evidence-bearing events.	Functional	Reference-labeled event benchmark
FR4	Select a target tier, operation, and recall scope independently from semantic extraction.	Functional	Routing and operation accuracy
FR5	Reject or escalate unsupported, sensitive, conflicting, or destructive durable-memory operations.	Safety	Policy-gate fixture tests
FR6	Perform scheduled deduplication, revalidation, archival, and garbage-collection candidacy without irreversible automatic deletion.	Functional	Controlled-clock maintenance tests
FR7	Build a minimum auditable Resume Pack linked to source memory records.	Functional	Recall precision, coverage, and trace audit
QR1	Preserve provenance from source event through final memory version and later recall.	Quality	Audit-chain completeness
QR2	Use idempotent queue processing and versioned writes so retries do not create duplicate canonical state.	Reliability	Duplicate-event replay
QR3	Keep the prototype readable and reproducible with local Markdown or JSON records and exported JSONL traces.	Maintainability	Artifact inspection and rerun

3.6 Proposed Architecture Workflow or Conceptual Model

The architecture has a storage plane and a control plane. The storage plane separates Project Memory, Daily Log, and Session Memory. The control plane moves information through Hook adapters, a Candidate Queue, an AI Curator, a Memory Router, a Policy Gate, optional human review, and a Versioned Writer. Cron invokes the same controlled operations on a schedule rather than creating a second maintenance path. The Resume Pack Builder reads the current stores but returns only a bounded set of records linked to their provenance.

The workflow begins with a concrete development event. A Hook records the event and evidence reference. The Curator proposes what the event means. The Router proposes where that meaning belongs and which operation is appropriate. The Policy Gate determines whether the proposal is sufficiently supported and permitted. Accepted writes create a new version; blocked or ambiguous proposals remain visible. Maintenance may later revalidate, deduplicate, archive, or identify garbage-collection candidates. When the agent resumes work, retrieval operates over the current versioned state and produces a compact package.

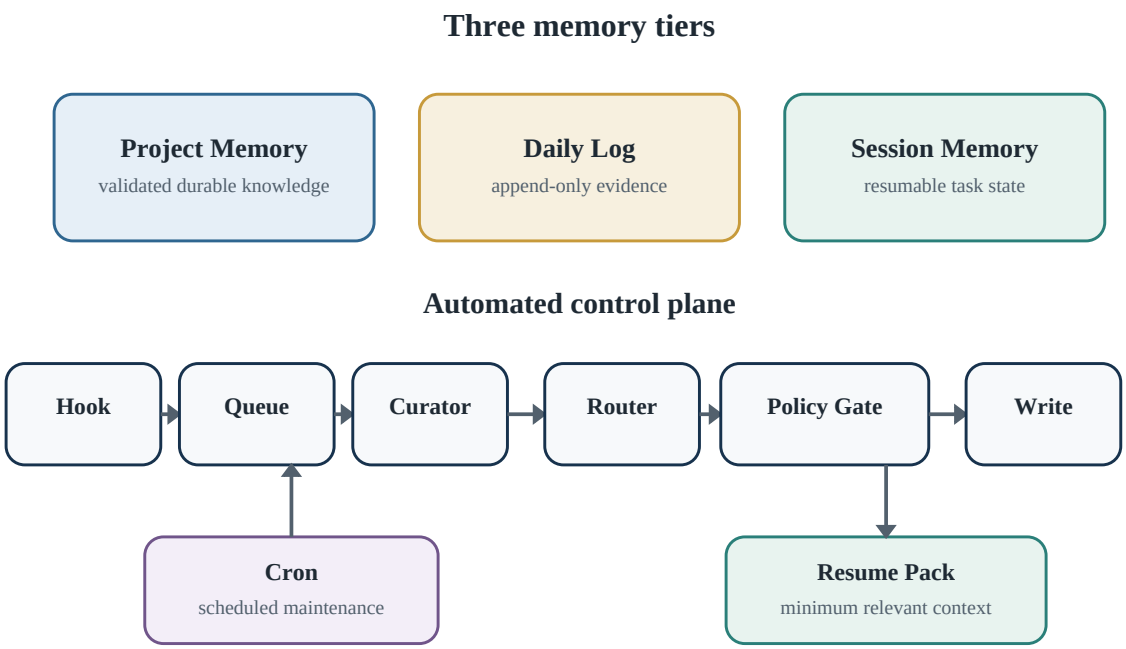


Figure 2. Automated three-tier external-memory architecture. Source: authors' proposed design.

Evidence remains linked from source event to resumed task

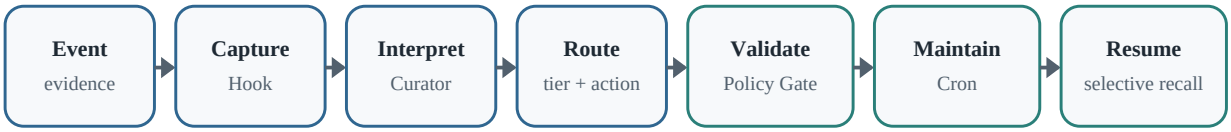


Figure 3. Evidence-aware memory lifecycle. Source: authors' proposed design.

3.7 Major Components

Table 3 describes component responsibilities and boundaries. The explicit Curator-Router separation is important. The Curator interprets semantic content, while the Router applies the approved memory taxonomy and chooses an action. This makes routing independently testable and avoids hiding placement decisions inside a single model response.

Table 5. Component Responsibilities and Safeguards

Component	Responsibility	Input	Output	Primary safeguard
Hook adapters	Observe approved lifecycle events and create candidates.	Task, command, test, correction, and session events	Idempotent event record	No direct canonical write
Candidate Queue	Buffer, retry, deduplicate, and sequence events.	Event records	Queued candidate with status	Deduplication key and retry log
AI Curator	Interpret semantic content and identify candidate claims.	Event plus evidence	Claims, uncertainty, conflict indicators	Evidence references required
Memory Router	Choose destination tier, operation, and recall scope.	Curated candidate	Routing proposal	Separate policy from interpretation
Policy Gate	Apply deterministic schema, provenance, secret, confidence, and destructive-action rules.	Routing proposal and current state	Accept, block, or review	Versioned rule set
Review Queue	Hold ambiguous or high-risk proposals for human judgment.	Escalated proposal	Approved, corrected, or rejected action	Reason and reviewer recorded
Versioned Writer	Apply atomic append, update, supersede, archive, or no-op operations.	Approved action	New memory version	Rollback and before/after snapshot
Cron Maintainer	Invoke routine maintenance at controlled checkpoints.	Memory state and schedule	Maintenance proposals and reports	Same gate as event-driven writes
Resume Pack Builder	Select the smallest relevant context for a new session.	Task request and current memory	Linked recall package	Token limit and provenance

3.8 Data Inputs Outputs and Interfaces

Events and decisions will use structured JSON-compatible records. Each event will carry a unique identifier, run and task identifiers, source type, source location or hash, capture time, and deduplication key. Candidate claims will include supporting evidence, uncertainty, and proposed conflict relationships. Memory records will include a tier, status, content hash, source commit, creation and verification time, and a supersedes link when applicable. Recall packages will list selected memory identifiers, fact units, relevance labels, token size, and retrieval time.

Human-readable Markdown views may be generated for inspection, but the authoritative experimental records will preserve structured fields. JSONL traces will make event order and decisions replayable. Interfaces will be local in the minimum prototype: command-line or test-harness invocation for event submission, review, maintenance, recall, and evaluation export.

3.9 Role of Artificial Intelligence

AI is appropriate for tasks that require semantic interpretation, such as extracting a candidate architectural decision from a mixed development trace, summarizing task progress, or identifying that two claims may conflict despite different wording. The AI Curator will receive the event, bounded surrounding evidence, relevant current memory, and an explicit schema. It will return proposed claims, evidence links, uncertainty, and conflict indicators. The Router may use model assistance for ambiguous semantic classification, but its output remains a proposal subject to policy.

AI will not directly commit canonical Project Memory, delete records permanently, bypass secret checks, or decide that a claim is true without evidence. Model outputs will be validated against schemas and policy, preserved in decision traces, and compared with reference labels. Model variability will be addressed through repeated event replays and a pinned configuration.

3.10 Human Oversight and Validation

Human review will focus on ambiguous, high-risk, or destructive cases rather than every routine event. Review triggers include insufficient evidence, unresolved conflicts, sensitive strings, promotion of uncertain claims into Project Memory, and proposed permanent deletion. Reviewers will see the source event, Curator interpretation, Router proposal, policy reason, and affected memory versions. Their decision and correction will be recorded.

The design does not equate fewer interventions with correctness. Review and recovery time will be counted as effort, and unnecessary escalations will be reported. Automated tests, schema validation, provenance checks, and memory-state postconditions provide additional verification.

3.11 Security Privacy and Ethical Considerations

The prototype will use a local fixture repository and synthetic task inputs. Secret-detection cases will contain nonfunctional dummy strings; real credentials will not enter logs or memory. Access to raw traces will be limited to the project team, and shared excerpts will be redacted. Minimum relevant context will be sent to any model provider. The system will prevent cross-project writes in the formal fixtures and will not run on confidential organizational repositories.

Auditability is a central ethical requirement because durable memory can influence later software changes. The system will preserve proposed, blocked, reviewed, and executed actions rather than reporting only successful final state. Automated permanent deletion remains outside prototype authority.

3.12 Traceability to the Problem and Literature

The architecture responds directly to literature findings. Hierarchical and tiered memory motivates separation of storage roles. Reflection and experiential learning motivate retaining failures and validation outcomes. Repository-memory research motivates durable project knowledge grounded in repository history. Studies of stale memory and faulty consolidation motivate provenance, versioning, revalidation, and deterministic policy checks. Benchmark limitations motivate an evaluation that isolates memory while holding the coding system constant.

Table 6. Problem Design and Evaluation Traceability

Problem element	Design response	Evaluation evidence
Users repeatedly decide what to save and reload	Hooks, scheduled maintenance, automatic Resume Packs	Manual actions and resumption delay
Tentative observations can become durable claims	Three tiers, evidence requirements, Policy Gate	Incorrect Project Memory write rate
Memory can become stale or conflicting	Version links, revalidation, conflict escalation	Stale-reference and conflict event cases
Long histories create context bloat	Router-selected minimum recall scope	Recall precision, coverage, token size
Unsafe automation may hide errors	Inspectable traces, review queue, no permanent automatic deletion	Integrity checks and audit-chain completeness
Memory may not improve software work	Connected coding-task comparison	Acceptance tests, regressions, repeated mistakes, repairs

3.13 Feasibility

The minimum implementation uses Python, Git snapshots, pytest, JSONL logging, Markdown or JSON memory files, a local queue, Hook adapters, a scheduler, and one pinned model interface. These technologies are accessible within the project environment and do not require production deployment or private data. The architecture is modular, so each component can be tested with fixed fixtures before integration.

The primary scope is feasible because it uses one repository, a limited event set, and a small number of paired task sequences. Optional ablations or additional baselines will proceed only after the required artifact and evaluation are complete.

3.14 Proposed Capstone II Artifact

The Capstone II artifact will include the prototype memory manager, three-tier schemas, lifecycle event adapters, a candidate queue, Curator and Router interfaces, deterministic policy validators, versioned storage, scheduled maintenance, Resume Pack generation, a sixty-case memory-event benchmark, a six-task connected coding fixture, automated tests, reproducible evaluation scripts, and technical documentation. The result will be an inspectable research prototype rather than a commercial service.

3.15 Chapter Summary

The proposed design assigns different responsibilities to three memory tiers and moves information through an evidence-aware control plane. AI provides bounded semantic interpretation; deterministic rules protect durable state; humans review exceptions; and scheduled maintenance reuses the same guarded operations. The architecture is directly connected to the problem, grounded in the literature, feasible within the course boundary, and testable through the methodology in Chapter 4.

Chapter 4 Evaluation and Research Methodology

4.1 Introduction

This chapter defines how Capstone II will produce credible evidence about the proposed memory lifecycle. The evaluation combines a component-event benchmark with a controlled sequence of related coding tasks. The first stage asks whether individual memory decisions are correct. The second asks whether those decisions reduce routine effort and help work continue across fresh agent sessions. All sample sizes, thresholds, and procedures are planned; no experimental result is claimed in this proposal.

4.2 Evaluation Purpose

The evaluation will determine whether lifecycle automation reduces manual memory work while maintaining memory integrity and software-task performance, with its additional cost made visible. This purpose requires more than a retrieval score. A system that saves effort by skipping all writes, sending every case to a human, or recalling large amounts of irrelevant context will not be considered successful.

4.3 Evaluation Questions

Four questions connect the problem, proposed system, and measurable evidence. EQ1 addresses user effort. EQ2 addresses component correctness. EQ3 addresses software-engineering outcomes. EQ4 exposes cost and operational risk.

Table 7. Evaluation Questions and Evidence

Question	Evidence needed
EQ1. Does automation reduce routine human memory work and resumption delay?	Save, load, review, correction, and recovery actions; active operator time; time to first validated progress.
EQ2. Does the pipeline capture, interpret, route, update, and retrieve memory correctly?	Reference-labeled events; proposed and executed operations; evidence links; memory snapshots; recalled fact units.
EQ3. Does automated memory preserve coding quality across related tasks?	Acceptance and regression tests; repeated failed approaches; repair cycles; violations of valid project constraints.
EQ4. What overhead and operational risks does automation introduce?	Model usage, elapsed time, storage growth, queue retries, maintenance actions, false escalations, and review burden.

4.4 Research and Evaluation Design

The study uses a controlled prototype evaluation with paired conditions. Both conditions use the same three-tier schema, evidence fields, storage representation, validators, repository fixtures, tasks, tests, tool permissions, recall limit, model version, and per-task resource ceiling. Condition A invokes the lifecycle through an explicit operator checklist. Condition B invokes the same functions through events and scheduled maintenance. Extra Curator or Router calls in B count toward total cost.

This design estimates the effect of the automation package rather than the isolated causal contribution of every component. The baseline is intentionally competent. It follows defined save and recall checkpoints,

so the comparison does not reward automation merely because the manual condition was neglected. Optional component ablations will remain exploratory.

Table 8. Primary Experimental Conditions

Condition	Lifecycle behavior
A Semi-manual three-tier baseline	The operator explicitly requests saving, session selection, and recall. The same tier definitions, evidence fields, storage format, and write safeguards remain available. Autonomous triggers and scheduled maintenance are disabled.
B Automated three-tier system	The same memory foundation uses Hooks, Candidate Queue, AI Curator, Memory Router, Policy Gate, exception review, Cron maintenance, and automatic Resume Packs.

4.5 Units of Analysis Cases and Test Subjects

Stage One contains sixty formal memory-event cases in ten families with six cases per family: stable facts, completed tasks and validation, unfinished session state, failed attempts, repeated events, conflicting claims, changed or stale references, explicit user corrections, synthetic sensitive strings, and irrelevant events that should be skipped. Twelve separate development cases will be used for instrumentation and rubric refinement and excluded from formal scores. The sixty formal cases will be replayed three times under a frozen configuration, producing 180 executions but only sixty distinct cases.

Stage Two uses a small local Python task-management service with session-cookie authentication, a service layer, SQLite storage, and pytest acceptance tests. Six connected tasks create controlled opportunities for persistent decisions, failed-attempt memory, resumption, drift, and regression. Each condition will run the full sequence three times. Tasks inside a sequence are dependent and will not be reported as independent experiments.

Table 9. Connected Coding Tasks

Task	Memory challenge	Verification
T1 Establish project constraints	Record authentication, service boundaries, and test commands from supplied evidence.	Expected memory facts and baseline tests
T2 Add task filtering	Reuse earlier constraints and internal service conventions.	Feature tests and regression suite
T3 Fix a permissions defect	Use supplied failed-patch evidence and avoid repeating the failed approach.	Permissions tests and repeated-mistake label
T4 Resume audit logging	Start a fresh conversation and recover unfinished work.	Progress checkpoint and audit-log tests
T5 Adapt to a controlled refactor	Detect outdated paths or interfaces while preserving valid facts.	Migration tests and freshness postconditions
T6 Extend task reporting	Apply current knowledge without breaking earlier behavior.	Reporting tests and full regression suite

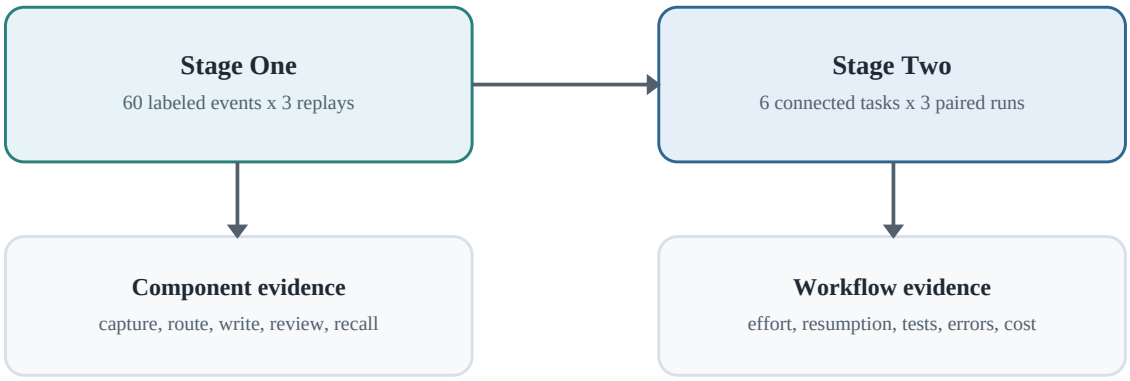


Figure 4. Two-stage evaluation design. Source: authors' planned study.

4.6 Data Sources

The study will preserve raw observations and derived measures. Every record will include a protocol version and identifiers that connect the source event, decision, memory change, recall package, and subsequent task. The collected state must distinguish what a model proposed from what the system actually executed. Missing usage values will be labeled unavailable instead of replaced with zero.

Table 11. Data Sources and Instruments

Dataset	Core fields and collection method
Run manifest	run ID, pair ID, condition, repository hash, task version, model configuration, budgets, clock schedule, operator, start and end time
Event and queue log	event, task, and session IDs; source type and hash; capture time; deduplication key; queue status; retry count
Decision trace	candidate claims, evidence references, uncertainty, proposed tier and operation, policy outcome, review reason, executed action
Memory versions	memory ID, tier, status, content and evidence hashes, source commit, verification time, supersedes link, before and after state
Recall package	request and task IDs, selected memory IDs, fact units, relevance labels, token size, retrieval time
Coding outcomes	acceptance and regression results, tool transcript, patch hash, repair cycles, repeated-error labels, stop reason
Human work log	action type, active time, reason, affected memory IDs, correction outcome
Usage and maintenance	component calls and tokens, durations, maintenance records scanned and changed, retries, storage size
Reference annotations	case ID, independent labels, rubric version, acceptable outcomes, disagreement, and adjudication

4.7 Variables Metrics and Measures

Metrics are operationally defined in Table 8. Effort and speed measures answer EQ1. Tier, operation, write, and recall measures answer EQ2. Tests, repeated mistakes, repairs, and regressions answer EQ3. Tokens, calls, storage, queue behavior, and review burden answer EQ4. Counts will accompany proportions so small denominators remain visible.

A repeated mistake reuses a previously failed approach under materially unchanged conditions despite available failure evidence. A repair cycle is one code revision followed by validation after an earlier failed validation. A regression is an earlier passing requirement that fails after a change. A context error violates a supplied project decision that remains valid. These definitions will be frozen before formal execution.

Table 10. Metrics and Decision Criteria

Measure	Operational definition	Proposed decision criterion
Manual memory actions	Explicit save, select, recall, review, correction, and recovery actions. Ordinary coding prompts are excluded.	At least 30 percent lower median sequence total in B; active human seconds also reported.
Resume delay	Seconds from fresh-session task delivery to the first predefined checkpoint showing valid progress, including retrieval and correction time.	At least 25 percent lower median sequence-level delay in B, without more failed resumptions.
Tier and operation correctness	Reference-compatible tier-operation outcomes divided by adjudicable formal cases.	At least 85 percent; final state after policy decisions reported separately.
Incorrect Project Memory writes	Unsupported, contradicted, wrong-scope, or unjustifiably promoted claims committed to Project Memory divided by committed Project Memory claims.	No more than 5 percent; blocked proposals reported separately.
Recall precision	Correct and task-relevant recalled fact units divided by all recalled fact units.	At least 80 percent.
Recall coverage	Required fact units present divided by the frozen task reference set.	At least 80 percent.
Software-task success	All mandatory acceptance and regression checks pass within budget.	Observed completion proportion in B no more than 5 percentage points below A.
Integrity checks	Committed synthetic secrets, unauthorized cross-project writes, and unapproved permanent deletions.	Zero occurrences; any occurrence blocks a favorable adoption decision.
Cost	Tokens, calls, latency, storage, review time, repairs, and maintenance effort.	Reported transparently; no pre-pilot monetary threshold.

4.8 Baseline or Comparison Method

Condition A is the primary baseline because the research question concerns automation of the lifecycle rather than the existence of the three-tier schema. The operator will use a written checklist at task boundaries, designated validation or correction events, and resumption. Review and correction rules remain the same in both conditions. Unrelated product memory will be disabled, and conditions will not share logs or memory.

Counterbalanced paired order will be A/B, B/A, and A/B. Each pair starts from isolated copies of the same initial fixtures. This ordering reduces, but does not eliminate, operator learning effects. Operator deviations will be logged.

4.9 Data Collection Instruments

Collection instruments include the event-case dataset, independent annotation rubric, operator checklist, run manifest, JSONL event and decision logs, versioned memory snapshots, recall-package records, pytest acceptance and regression tests, human-action logger, model-usage collector, controlled clock, and evaluation scripts. The appendices define the planned schemas and protocol steps.

4.10 Evaluation Procedure

First, the team will implement the fixture repository, event cases, hidden acceptance checks, annotation rubric, operator checklist, and instrumentation. Acceptance checks will be fixed before either condition is evaluated. Second, separate pilot cases and a short pilot sequence will calibrate model budgets, trigger rules, scheduler checkpoints, and scoring instructions. The protocol will then be versioned and frozen.

Third, formal component cases will run from their specified starting states. Proposed actions, blocked actions, executed changes, and evidence will be retained. Fourth, the six-task coding sequence will run three times per condition using paired starting states. Fresh agent conversations will begin at every task boundary. Fifth, the actual maintenance routine will be invoked through a controlled clock at fixed checkpoints, while a separate smoke test will verify that the scheduler invokes it. Finally, tests and the frozen rubric will score outputs. Pilot data will remain separate.

4.11 Data Management

UTC timestamps will align traces and monotonic timers will measure duration. Content hashes and identifiers will link records without exposing credentials. Raw observations, decisions, snapshots, and derived summaries will be stored in versioned local project directories with read access limited to the group. Shared appendices will omit sensitive raw traces. The proposed retention period is through final assessment and any applicable appeal period, subject to course requirements.

Snapshots will be taken at initial state, each accepted write, each task boundary, each maintenance checkpoint, and immediately before recall. Archival must preserve provenance and remain recoverable. Permanent automatic deletion is outside system authority.

4.12 Data Analysis Plan

Component analysis will report correct counts, denominators, percentages, and variation across three replays by event family. Tier-operation errors and required-versus-observed review decisions will be tabulated. Initial annotator agreement and disagreements will be reported before adjudication. These deliberately selected fixtures describe coverage of the benchmark, not population-wide error rates.

For each complete coding sequence, the analysis will compute manual actions, human active seconds, completed tasks, failed resumptions, successful resume delays, repairs, regressions, repeated mistakes, and total cost. The report will show the three paired A/B results, paired differences, medians, and ranges. The thirty-six task attempts are nested inside six sequences and will not be treated as thirty-six independent experiments.

For effort or time, descriptive reduction equals 100 multiplied by the difference between the median of A and the median of B, divided by the median of A. When the baseline is zero, the absolute difference will be reported and relative reduction will be undefined. Cost will be separated by coding agent, Curator, Router, retrieval, and maintenance. Failure analysis will trace selected cases from event through decision, memory, recall, and coding outcome. Co-occurrence will not be treated as proof of causation.

4.13 Success Criteria

The study will support a promising feasibility finding within the tested setting only if the effort and resumption targets, component-quality targets, observed task-quality guardrail, and integrity checks are met. Cost must also be reported. If effort improves while quality degrades, the conclusion will identify an automation-quality trade-off rather than a successful design. Incomplete logging, substantial pair-to-pair disagreement, or unassessable integrity metrics will produce an inconclusive finding.

The thresholds are project decision criteria, not published standards. Any pilot-driven revision will be documented before formal collection. With three pairs, no confirmatory significance test, formal non-inferiority conclusion, or universal effectiveness claim is planned.

4.14 Reliability and Validity

Reliability protections include fixed fixtures, versioned protocols, controlled starting states, repeated event replays, independent case annotation, a frozen rubric, identical resource ceilings, and preserved execution traces. Both group members will label formal cases independently. Disagreements will be recorded and adjudicated before system execution; unresolved cases will be reported separately.

Data-quality checks will verify required identifiers, event ordering, evidence links, agreement between recorded writes and snapshots, and reconciliation between task summaries and test logs. Scoring records will use neutral run identifiers where feasible, although complete blinding is unlikely because automation traces reveal the condition.

4.15 Threats to Validity

Internal validity is threatened by operator learning, model variability, protocol deviations, propagated task failures, and researcher influence. Counterbalanced order, pinned configurations, shared operator rules, independent labeling, and preserved deviations reduce these risks. A failed earlier task will remain part of the sequence rather than being silently replaced by a successful reference patch.

Construct validity is limited by the difficulty of reducing memory quality to labels and metrics. Reference sets will allow multiple acceptable tier-operation outcomes where appropriate. A passing test supports only the behavior tested. External validity is limited by one small repository, one model configuration, six tasks, controlled events, and simulated maintenance checkpoints. The study cannot establish production reliability or generalization across organizations.

4.16 Ethical Privacy and Security Considerations

The study uses local fixtures and synthetic inputs. No outside participants or private organizational repositories are planned. Dummy secret strings will be nonfunctional. Actual credentials will remain outside the repository, logs, and memory stores. Context sent to an external model will be minimized and governed by the selected service configuration. Any future addition of participants or private data would require review of institutional permissions before collection.

4.17 Limitations

The evaluation is a preliminary feasibility study. It cannot isolate every component's causal effect, establish a statistically powered non-inferiority margin, or determine how the architecture behaves over months of real development. The team members' roles as designers, operators, and annotators may influence fixtures and scoring. Controlled tasks may make useful memories more explicit than natural projects do. These limitations will be reported with the results rather than hidden by aggregate success measures.

4.18 Chapter Summary

The evaluation combines inspectable component cases with connected coding tasks. It compares automation against a competent semi-manual version of the same memory store, preserves an audit chain from event to task outcome, and interprets results conservatively. The planned metrics, data sources, procedures, and validity protections are sufficient to test feasibility while respecting the project's limited scale.

Chapter 5 Capstone II Execution Plan

5.1 Introduction

This chapter converts the proposal into an executable Capstone II plan. The work is divided into design finalization, prototype implementation, fixture and protocol preparation, pilot calibration, formal collection, analysis, and reporting. Required work is separated from optional enhancement so that the core research question remains achievable within one term.

5.2 Work Breakdown

The first work package finalizes schemas, requirements, and fixtures. The second implements storage and deterministic infrastructure. The third integrates AI interpretation, routing, policy, maintenance, and recall. The fourth prepares and labels evaluation cases. The fifth pilots and freezes the protocol. The sixth executes formal collection. The seventh analyzes evidence. The final package documents the system and results.

Each package produces a versioned deliverable and acceptance gate. Optional work, including additional repositories, a no-memory baseline, user interface, learned routing, or long-duration deployment, will begin only if the primary artifact and study are complete.

5.3 Capstone II Milestones

The major milestones are an approved design baseline, a tested three-tier store, an integrated automated lifecycle, a frozen evaluation protocol, a complete formal dataset, an audited analysis, and a reproducible final artifact. The schedule in Table 11 associates each milestone with a concrete acceptance condition rather than a completion claim based only on elapsed time.

5.4 Schedule

The proposed fourteen-week schedule leaves the final three weeks for formal analysis, documentation, and advisor feedback. Pilot and formal cases remain separate. If implementation slips, optional features will be removed before collection rather than changing the research question after results are observed.

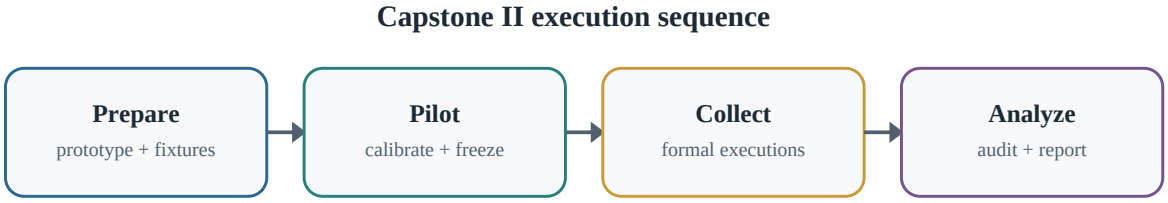


Figure 5. Capstone II execution sequence. Source: authors' planned schedule.

Table 12. Capstone II Schedule and Milestones

Phase and weeks	Primary work	Deliverable and acceptance gate
Phase 1 Weeks 1-2	Finalize requirements, schemas, fixture repository, event taxonomy, and threat model.	Versioned design package; baseline tests pass; formal fixtures are separate from pilot fixtures.
Phase 2 Weeks 3-5	Implement three stores, Hook adapters, queue, versioned writer, and structured trace logging.	Component tests pass; retries are idempotent; snapshots match recorded writes.
Phase 3 Weeks 6-7	Implement Curator, Router, Policy Gate, review flow, Cron maintenance, and Resume Pack Builder.	End-to-end lifecycle runs locally with inspectable evidence and no direct Hook-to-memory write.
Phase 4 Week 8	Complete sixty formal cases, twelve pilot cases, annotation rubric, hidden tests, and operator checklist.	Both annotators complete labels; disagreements are resolved or explicitly marked.
Phase 5 Week 9	Run pilot cases and pilot coding sequence; calibrate budgets and thresholds; freeze protocol.	Protocol, model configuration, fixtures, and evaluation scripts receive immutable version tags.
Phase 6 Weeks 10-11	Execute formal event replays and three paired six-task sequence repetitions.	All attempts, failures, memory states, usage records, and deviations are preserved.
Phase 7 Week 12	Audit data, compute descriptive comparisons, trace failures, and prepare result displays.	Every result links to raw evidence, denominator, protocol version, and limitation.
Phase 8 Weeks 13-14	Complete technical documentation, final report, presentation, and artifact package.	Reproducible package passes acceptance checklist and advisor review.

5.5 Required Resources

The project relies on widely available local development tools and synthetic data. Model access is required for the Curator and may be required for ambiguous routing, but deterministic fixture tests will cover queue, policy, versioning, maintenance, and trace behavior without depending on a live model. No external participants, production services, or commercial-scale infrastructure are required.

Table 13. Required Resources

Resource	Planned use	Availability and fallback
Python and pytest	Prototype, fixtures, controlled tasks, and automated checks	Local and open source; minimal dependencies preferred
Git	Repository state, hashes, branches, and reproducible snapshots	Local repository; export tags with artifact
Markdown JSON and JSONL	Human-readable memory plus structured records and traces	Local files; schema-versioned
Pinned model interface	Curator and limited routing interpretation	Provider or local option to be fixed before pilot; deterministic fallback fixtures for non-model tests
Local queue and scheduler	Idempotent processing and controlled Cron invocation	Standard library or lightweight package; controlled clock for tests
Fixture repository	Six connected coding tasks and hidden checks	Created locally with synthetic data
Analysis scripts	Counts, proportions, paired differences, medians, ranges, and tables	Python standard analysis stack; all calculations scripted

5.6 Technical Environment

The planned environment is a local Python project under Git version control. Pytest will run unit, integration, acceptance, and regression checks. Structured JSON or JSONL will record events and decisions; Markdown or JSON will represent memory. A lightweight queue and scheduler will invoke reusable lifecycle functions. A controlled clock will reproduce maintenance checkpoints. Analysis scripts will calculate descriptive results and produce tables from preserved data.

5.7 Dependencies

Formal collection depends on completed instrumentation, stable fixture tests, independent case labels, a frozen protocol, and a pinned model configuration. Stage Two also depends on the event-driven system passing critical Stage One integrity tests. Provider access and budget must be confirmed before the pilot. If the preferred model becomes unavailable, the team will select and pin a replacement before protocol freeze rather than switching during formal collection.

5.8 Risks and Mitigation

The risk register emphasizes threats that could invalidate conclusions rather than only delay implementation. Missing logs, an unfair baseline, or sensitive-data exposure are high-impact risks. The mitigation strategy preserves evidence, limits scope, freezes procedures, and defines when data must be treated as unassessable.

Table 14. Risk Register

Risk	Probability	Impact	Trigger	Mitigation and contingency
Model output varies across runs	High	Medium	Different claim or routing proposal for the same fixture	Pin configuration, repeat cases, preserve traces; use deterministic rules for safety-critical checks.
Scope exceeds the term	Medium	High	Required modules or fixtures unfinished by Week 7	Protect the minimum prototype; defer optional ablations, UI, and additional repositories.
Labels are ambiguous	Medium	Medium	Frequent annotator disagreement or unscorable cases	Allow acceptable outcome sets, freeze rubric, report unresolved cases separately.
Baseline is not comparable	Low	High	Different schema, model, limits, or validation between conditions	Use one shared implementation and invoke functions manually or automatically; audit run manifests.
Logging is incomplete	Medium	High	Missing event-to-write or recall-to-task link	Treat metrics as unassessable; stop collection, repair instrumentation, rerun with original record retained.
Provider cost or access changes	Medium	Medium	Budget ceiling or model availability changes	Use a pinned alternative, reduce repeats only with documented protocol amendment, report usage rather than price.
Synthetic tasks are too easy	Medium	Medium	Both conditions achieve ceiling performance	Use pilot-only calibration and memory-only historical decisions; retain task-success guardrails.
Sensitive content enters logs	Low	High	Secret scanner flags nonfixture content	Use dummy secrets, local fixtures, pre-share redaction, and no private repositories.

5.9 Quality Assurance

Quality assurance will combine unit tests, schema validation, idempotency tests, integration tests, fixture postconditions, code review, independent annotation, trace audits, and reproducibility checks. Formal data collection will begin only after the pilot confirms that identifiers, snapshots, timers, and usage records reconcile. Hidden acceptance tests will be fixed before either condition is run.

Every reported result will retain its denominator and link to a protocol version. Failed and inconvenient runs will remain in the dataset. The team will review diagrams, documentation, and report tables against the actual implementation before submission.

5.10 Configuration and Version Management

Git will version code, schemas, prompts, fixtures, tests, rubrics, diagrams, and analysis scripts. Formal run manifests will record repository hashes, model configuration, budgets, and task versions. Memory snapshots and logs will be stored under run-specific identifiers. Changes after protocol freeze will require a written amendment; original artifacts and affected runs will remain available.

5.11 Documentation Plan

The project will maintain an architecture description, component contracts, memory schema guide, prompt and policy versions, event-case catalog, operator checklist, evaluation protocol, data dictionary, risk register, run ledger, analysis notebook or script documentation, user guide, and final report. Documentation will distinguish design intent from observed behavior.

5.12 Completion and Acceptance Criteria

Capstone II will be complete when the bounded prototype is implemented, three tiers and the guarded lifecycle operate end to end, critical component and integrity tests pass, the sixty formal event cases and paired task sequences are executed or transparently accounted for, the data is audited, comparative results and failures are analyzed, threats and limitations are documented, and the code, fixtures, protocol, results, user guide, final report, and presentation are packaged under version control.

The project is not defined as complete merely because a demonstration runs. It must produce inspectable evidence sufficient to answer the evaluation questions, including negative or inconclusive findings.

5.13 Chapter Summary

The work plan decomposes the project into measurable phases with explicit dependencies, risks, and acceptance gates. The required tools and data are accessible, and optional enhancements are isolated from the minimum study. The plan is therefore ready to guide Capstone II implementation and evaluation.

Chapter 6 Expected Contributions and Conclusion

6.1 Proposal Summary

This proposal addresses the loss and fragmentation of useful project context across coding-agent sessions. The literature supports external memory, hierarchy, reflection, adaptive operations, repository history, and experience reuse, but it also documents stale guidance, harmful consolidation, over-direction, and context cost. These findings motivate a lifecycle that separates memory roles, preserves evidence, constrains AI authority, and evaluates the consequences of automation.

The proposed response is an automated three-tier external-memory system. Project Memory stores stable validated knowledge, the Daily Log preserves chronological evidence, and Session Memory maintains one resumable task. Hooks, a queue, an AI Curator, a separate Router, a deterministic Policy Gate, versioned writes, scheduled maintenance, review, and selective Resume Packs control information movement.

6.2 Expected Artifact or Research Outcome

At the end of Capstone II, the project is expected to deliver a local working prototype, memory schemas, a controlled lifecycle implementation, a memory-event benchmark, a connected coding-task fixture, evaluation scripts, preserved experimental traces, an analysis report, and technical documentation. The artifact is expected to make memory decisions auditable from source event to later recall.

6.3 Expected Contribution to Practice

The expected practical contribution is an implementable pattern for reducing routine save and reload work without granting an LLM unrestricted control over durable project knowledge. The architecture may help small teams and developers distinguish stable project facts from evidence and temporary work, decide which checks should remain deterministic, and make scheduled memory maintenance recoverable.

6.4 Expected Contribution to Knowledge

The expected research contribution is preliminary evidence about the effort, quality, integrity, and cost trade-offs of lifecycle automation in one controlled coding setting. The study also contributes an operational taxonomy of memory-event cases, explicit metric definitions, and an evaluation protocol that holds the memory foundation constant while changing how the lifecycle is invoked.

These contributions are anticipated rather than completed. A negative or inconclusive result would still identify failure mechanisms and measurement limitations relevant to later repository-memory studies.

6.5 Anticipated Limitations

The expected evidence will be limited by one repository, one model configuration, controlled tasks, three paired sequences, simulated elapsed time for maintenance, and researcher involvement in design and scoring. The study will not establish long-term production reliability, broad generalization, enterprise governance, or the independent causal effect of every component. Several directly relevant sources are recent preprints, so the technical foundation may evolve.

6.6 Readiness for Capstone II

The project is ready to proceed because the problem and scope are stable, the literature has been synthesized, the solution has defined components and boundaries, the evaluation identifies baselines and operational metrics, the data plan preserves an audit chain, and the execution plan specifies resources, risks, milestones, and acceptance criteria. Remaining implementation choices, including the exact model interface and lightweight queue package, will be fixed before the pilot and recorded in the protocol.

6.7 Conclusion

Long-horizon coding work requires continuity that temporary model sessions do not provide by themselves. An external memory system can help only if it preserves evidence, limits incorrect durable writes, controls context cost, and demonstrates value in software tasks. The proposed three-tier architecture and controlled evaluation provide a feasible way to test that claim in Capstone II without assuming the result in advance.

References

Bae, S., Lee, J., Choi, C., Choi, Y., & Lee, J.-H. (2026). Remember your trace: Memory-guided long-horizon agentic framework for consistent and hierarchical repository-level code documentation. *arXiv preprint arXiv:2605.14563*.

Bairi, R., Sonwane, A., Kanade, A., D C, V., Iyer, A. S., Parthasarathy, S., Rajamani, S., Ashok, B., & Shet, S. (2024). CodePlan: Repository-level coding using LLMs and planning. *Proceedings of the ACM on Software Engineering, 1*(FSE), 675-698.

Chao, H., Bai, Y., Sheng, R., Li, T., & Sun, Y. (2026). STALE: Can LLM agents know when their memories are no longer valid?. *arXiv preprint arXiv:2605.06527*.

Chen, Z., Tang, R., Deng, G., Wu, F., Wu, J., Jiang, Z., Prasanna, V., Cohan, A., & Wang, X. (2025). LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* (pp. 8697-8727).

dos Santos, H. V. F., Costa, V., Montandon, J. E., Silva, L. L., & Valente, M. T. (2026). Configuration smells in AGENTS.md files: Common mistakes in configuring coding agents. *arXiv preprint arXiv:2606.15828*.

Gao, H., & Zhang, Y. (2024). Memory sharing for large language model based agents. *arXiv preprint arXiv:2404.09982*.

Gao, S., Zeng, W., Yu, Z., Wangni, J., Wang, C., Cai, K., He, S., & Lyu, M. R. (2026). SWE-MeM: Learning adaptive memory management for long-horizon coding agents. *arXiv preprint arXiv:2606.28434*.

Gloaguen, T., Mündler, N., Müller, M., Raychev, V., & Vechev, M. (2026). Evaluating AGENTS.md: Are repository-level context files helpful for coding agents?. *arXiv preprint arXiv:2602.11988*.

Hu, M., Chen, T., Chen, Q., Mu, Y., Shao, W., & Luo, P. (2025). HiAgent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* (pp. 32779-32798).

Hu, Y., Liu, S., Yue, Y., et al. (2025). Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*.

Hu, Y., Wang, Y., & McAuley, J. (2026). Evaluating memory in LLM agents via incremental multi-turn interactions. In *International Conference on Learning Representations*.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues?. In *International Conference on Learning Representations*.

Li, M., Xu, L. H., Tan, Q., Cao, T., & Liu, Y. (2026). Learning to commit: Generating organic pull requests via online repository memory. *arXiv preprint arXiv:2603.26664*.

Lu, M., Wu, M., Liu, F., Xu, J., Li, W., Wang, H., Hu, Z., Ding, Y., Sun, Y., Lu, J., & Zhang, Y. (2026). Choosing how to remember: Adaptive memory structures for LLM agents. *arXiv preprint arXiv:2602.14038*.

Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., & Fang, Y. (2024). Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.

Park, J. S., O'Brien, J., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (Article 2, pp. 1-22).

Qian, C., Dang, Y., Li, J., Liu, W., Xie, Z., Wang, Y., Chen, W., Yang, C., Cong, X., Che, X., Liu, Z., & Sun, M. (2024). Experiential co-learning of software-developing agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics* (pp. 5628-5640).

Reddy, V., & Challaram, S. (2026). Don't ask the LLM to track freshness: A deterministic recipe for memory conflict resolution. *arXiv preprint arXiv:2606.01435*.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems, 36*.

Wang, B., Xu, W., Li, Y., Gao, M., Xie, Y., Sun, H., & Chen, D. (2026). Improving code localization with repository memory. In *International Conference on Learning Representations*.

Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., & Yu, D. (2025). LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations*.

Xiong, Z., Lin, Y., Xie, W., He, P., Liu, Z., Tang, J., Lakkaraju, H., & Xiang, Z. (2026). How memory management impacts LLM agents: An empirical study of experience-following behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics* (pp. 623-645).

Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., & Zhang, Y. (2025). A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*.

Xu, W., Wang, Y., Mei, K., Liang, K., Wang, Z., Jin, M., Zhang, H., Zhang, S.-X., Hua, W., Sahu, S., & Metaxas, D. N. (2026). MemGym: A long-horizon memory environment for LLM agents. *arXiv preprint arXiv:2605.20833*.

Yan, S., Yang, X., Huang, Z., Nie, E., Ding, Z., Li, Z., Ma, X., Schütze, H., Tresp, V., & Ma, Y. (2025). Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems, 37*.

Yu, Y., Yao, L., Xie, Y., Tan, Q., Feng, J., Li, Y., & Wu, L. (2026). Agentic memory: Learning unified long-term and short-term memory management for large language model agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics* (pp. 21457-21483).

Zhang, D., Lin, Y., Wu, Z., Sun, Y., Li, B., Li, D., & Peng, H. (2026). Useful memories become faulty when continuously updated by LLMs. *arXiv preprint arXiv:2605.12978*.

Zhang, Y., Shu, J., Ma, Y., Lin, X., Wu, S., & Sang, J. (2026). Memory as action: Autonomous context curation for long-horizon agentic tasks. *Findings of the Association for Computational Linguistics: ACL 2026*, 19149-19164.

Zhang, Z., Dai, Q., Bo, X., Ma, C., Li, R., Chen, X., Zhu, J., Dong, Z., & Wen, J.-R. (2025). A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems, 43*(6), Article 155.

Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y.-J., & Huang, G. (2024). ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence, 38*(17), 19632-19642.

Zhong, W., Guo, L., Gao, Q., Ye, H., & Wang, Y. (2024). MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence, 38*(17), 19724-19731.

Appendix A Literature Synthesis Matrix

This matrix preserves the common comparison fields used during the Week 7 synthesis. Peer-reviewed and preprint status should be considered when interpreting the findings.

Source	Evidence status	Method	Finding	Limitation and relevance
Zhang et al. (2025)	Peer-reviewed journal article	The authors conduct a structured literature survey and organize memory by information source, representation form, memory operation, and evaluation strategy.	Memory should be treated as a module that collects, processes, stores, and later uses information, rather than as retained chat history alone.	The taxonomy spans many agent domains and does not determine a coding-specific schema, lifecycle, or implementation. It provides the strongest peer-reviewed foundation for defining external memory and for structuring our lifecycle questions.
Park et al. (2023)	Peer-reviewed conference paper	The paper implements a sandbox with 25 agents, a natural-language experience stream, relevance-recency-importance retrieval, periodic reflection, and planning; it uses human evaluation and ablation.	Observation, reflection, and planning jointly produced more believable individual and emergent behavior than ablated variants.	Believability in a social simulation is subjective and does not measure repository correctness, tests, or codebase drift. It establishes the influential pattern of retaining raw episodes while generating higher-level abstractions for later action.
Shinn et al. (2023)	Peer-reviewed conference paper	Reflexion converts external or simulated feedback into verbal reflections stored in an episodic buffer and evaluates later trials on decision-making, reasoning, and HumanEval coding tasks, including ablations.	Textual reflection improved later behavior without weight updates; the paper reports 91 percent pass@1 on HumanEval.	A generated reflection can be incorrect, and HumanEval does not represent continuing work in one evolving repository. It directly supports storing failed attempts and validation feedback as external episodic memory.
Zhao et al. (2024)	Peer-reviewed conference paper	ExpeL autonomously gathers experiences from training tasks, extracts natural-language insights, and retrieves both similar trajectories and general lessons for unseen decision-making tasks.	Performance improved as useful experience accumulated, and the authors observed transfer of extracted lessons to new tasks.	The extraction process may turn mistaken interpretations into durable lessons and is not grounded in repository versions or tests. It motivates cross-task learning while highlighting the need for write-quality and evidence checks in coding memory.
Packer et al. (2023)	Preprint	MemGPT implements operating-system-inspired virtual context management with model-visible memory, external archival memory, recall storage, and explicit function calls for moving information.	The prototype demonstrates that an agent can manage information beyond its immediate context in document analysis and multi-session conversation settings.	As a preprint and broad architecture demonstration, it offers limited coding-specific evidence and no repository write-trust rule. It supplies the hierarchy and paging analogy behind separating active trajectory memory from larger durable stores.
Xu et al. (2025)	Preprint	A-MEM creates Zettelkasten-inspired notes with descriptions, keywords, tags, embeddings, and links; adding a note can revise attributes and relationships of older notes. It is evaluated with six foundation models and memory benchmarks.	Dynamic linked-note organization outperformed reported fixed storage and retrieval baselines across evaluated models.	Memory evolution is itself LLM-mediated, so provenance, correction, and protection from faulty updates remain unresolved. It supports structured, evolving memory but strengthens our case for transparent and reversible update policies.
Hu et al. (2025)	Peer-reviewed conference paper	HiAgent groups working memory by subgoal and allows completed subgoal traces to be replaced with summaries while current evidence remains detailed. It is tested on five long-horizon tasks.	The authors report a twofold success-rate increase and 3.8 fewer steps on average, with improvements across different task lengths.	The system manages in-trial working memory rather than durable, cross-session repository knowledge. It provides strong evidence for task-aware hierarchy and helps us separate trajectory compression from repository memory.
Yu et al. (2026)	Peer-reviewed conference paper	AgeMem exposes store, retrieve, update, summarize, and discard as agent tools and learns a unified policy through three-stage reinforcement learning with step-wise GRPO.	Across five long-horizon benchmarks and multiple model backbones, the learned policy improved task performance, memory quality, and context efficiency over memory-augmented baselines.	The training pipeline is expensive and optimizes benchmark reward, not necessarily repository truth, auditability, or team governance. It confirms that memory timing can be part of agent policy while supporting a simpler, auditable rule-based Capstone prototype.
Zhang et al. (2026a)	Peer-reviewed conference paper	MemAct treats context insertion and deletion as learnable policy actions and uses end-to-end reinforcement learning with Dynamic Context Policy Optimization.	The reported 14B model matched the accuracy of models 16 times larger while reducing average context length by 51 percent.	The method edits working context, requires specialized training, and does not provide durable repository provenance or drift checks. It supports explicit memory actions but clarifies why our artifact should focus on lifecycle transparency rather than model training.

Source	Evidence status	Method	Finding	Limitation and relevance
Xiong et al. (2026)	Peer-reviewed conference paper	The study experimentally manipulates memory addition and deletion, measures similarity between current tasks, retrieved experiences, and outputs, and analyzes later performance.	Agents exhibit experience-following: similar retrieved memories produce similar outputs, enabling both error propagation and misaligned replay. Later task results can label memory quality.	The controlled settings do not provide repository-specific truth checks or a complete safe memory policy. It shows that stored memory is a behavioral control input and justifies evidence-aware writes, deletion, and repeated-failure tracking.
Chao et al. (2026)	Preprint	STALE constructs 400 expert-validated scenarios and 1,200 queries covering state resolution, premise resistance, and implicit policy adaptation, then evaluates frontier models and memory systems.	The best evaluated system reached only 55.2 percent overall; retrieving updated evidence did not guarantee behavior consistent with the update.	The scenarios concern personal and everyday states rather than repositories, code dependencies, or executable validation. It supplies direct evidence for a memory-drift problem and a model for testing invalidation beyond exact string replacement.
Zhang et al. (2026b)	Preprint	The authors use a controlled ARC-AGI Stream environment with Retain, Delete, and Consolidate actions and compare update schedules, forced consolidation, no memory, and episodic-only controls.	Consolidated memory utility rose and then degraded, sometimes below no memory; preserving raw episodes matched or exceeded forced consolidation and doubled accuracy in a reported comparison.	ARC-AGI reasoning differs from coding, and the study is a very recent preprint awaiting replication. It strongly supports preserving raw evidence, gating consolidation, and making summaries reversible.
Gloaguen et al. (2026)	Preprint and workshop paper	The study compares no file, LLM-generated files, and developer-committed files across multiple models and coding agents on SWE-bench tasks and repositories with real context files.	Context files did not generally improve success and raised average inference cost by more than 20 percent, although agents followed the instructions.	Effects depend on file quality, repository, model, and task, and the study does not evaluate a selective dynamic memory lifecycle. It provides a necessary counterexample and supports minimal static guidance as a baseline rather than assuming more context is better.
Wang et al. (2026)	Peer-reviewed conference paper	The system mines recent commits, linked issues, and functionality summaries into non-parametric memory, adds retrieval tools to LocAgent, and evaluates on SWE-bench Verified and SWE-bench Live.	Repository memory significantly improves the strong localization framework, showing that history contributes information beyond the current code snapshot.	The evaluation concerns localization, not a complete build-test-repair lifecycle, cross-task updates, or team-shared governance. It is the strongest direct academic evidence for repository memory and anchors our coding-specific research gap.
Gao et al. (2026)	Preprint	SWE-MeM provides a memory tool for deciding when, what, and how to compress, then trains agents with synthesized trajectories and memory-aware GRPO on SWE-bench Verified.	The paper reports 43.4 and 60.2 percent resolve rates for 4B and 30B models, respectively, while improving efficiency over static memory-management baselines.	It is a recent preprint, jointly optimizes issue resolution and compression, and focuses on current-task trajectory memory. It directly validates adaptive management in coding but leaves durable, evidence-grounded repository memory unresolved.
Bae et al. (2026)	Preprint	MemDocAgent traverses dependencies and uses a shared RepoMemory with read, write, and verify operations so later components can reuse prior traces.	The reported evaluation shows stronger hierarchical consistency and repository-level documentation quality than baselines.	The task is documentation rather than code modification, and concurrent writes, permissions, and provenance are not deeply tested. It demonstrates pyramid-style shared repository memory and motivates separate file, module, and project representations.
Li et al. (2026)	Preprint	Using a strict chronological split, the agent attempts historical issues, compares attempts with oracle diffs, distills repository skills, and evaluates on genuinely later merged pull requests.	Online repository memory improves reported functional correctness, style consistency, internal API reuse, and modified-region plausibility on held-out future tasks.	The study uses one expert-maintained repository and oracle historical diffs that are unavailable for new live tasks. It shows that conventions indirectly related to a current issue can still be valuable durable project memory.
Qian et al. (2024)	Peer-reviewed conference paper	Instructor and assistant agents co-track trajectories, construct procedural graphs, mine shortcut-oriented experiences, and reuse them on unseen software-development tasks.	The extensive experiments report improved quality and efficiency on unseen tasks and fewer repeated inefficient attempts.	The multi-agent development setting does not deeply examine shared memory ownership, review, privacy, or rollback. It provides peer-reviewed evidence that shared software experience can help while exposing a governance gap for team memory.
Hu et al. (2026)	Peer-reviewed conference paper	MemoryAgentBench reformulates existing datasets and creates new tasks to evaluate accurate retrieval, test-time learning, long-range understanding, and selective forgetting under a unified incremental protocol.	No evaluated system masters all four competencies; selective forgetting and long-range understanding remain particularly weak.	The benchmark centers assistant interactions rather than evolving repositories, executable tests, or memory created during coding. It broadens our evaluation beyond recall and informs memory-quality and forgetting measures for the Capstone.

Source	Evidence status	Method	Finding	Limitation and relevance
Xu et al. (2026)	Preprint	MemGym unifies five tracks across four agentic regimes, creates memory-isolated scores, and uses ablation-verified synthetic pipelines; a lightweight reward model approximates some coding rollouts.	The framework makes memory contribution more separable from reasoning, retrieval, and tool use and exposes weak transfer from conversation-oriented memory systems.	It is a new preprint, and some coding evaluation uses a learned reward-model proxy rather than full executable repository rollouts. It directly motivates a controlled, task-sequence benchmark that varies memory while holding the coding agent constant.

Appendix B Planned Data Dictionary

Element	Type	Source	Purpose and sensitivity
event_id	String	Hook or fixture	Links a development event through the audit chain; non-sensitive identifier
source_hash	String	File, command, test, or user evidence	Confirms evidence version; may indirectly identify a local artifact
candidate_claim	Text	AI Curator	Proposed semantic content; may be wrong and is never proof by itself
proposed_tier	Enum	Memory Router	Project, Daily Log, Session, review, or skip
proposed_operation	Enum	Memory Router	Append, update, supersede, archive, no-op, or review
policy_outcome	Enum	Policy Gate	Accepted, blocked, or review required
memory_version	String	Versioned Writer	Connects before and after state and supports rollback
recall_fact_unit	Text	Resume Pack Builder	Unit scored for correctness, relevance, and coverage
human_action	Record	Operator interface	Manual save, recall, review, correction, or recovery action with active time
test_outcome	Record	pytest runner	Acceptance or regression result linked to a task attempt
usage_record	Record	Model interface and local timers	Calls, tokens if available, latency, and component identity

Appendix C Evaluation Protocol

1. Prepare versioned repository and memory snapshots, model configuration, task instructions, tests, budgets, controlled clock, and operator assignments.

2. Verify critical instrumentation by reconciling event, queue, decision, memory, recall, human-action, and test records.

3. Complete independent labels for formal memory-event cases and freeze the annotation rubric before system execution.

4. Run development-only pilot cases and a separate pilot task sequence; document any protocol changes and freeze the final version.

5. Replay each of the sixty formal event cases three times from its specified starting state; preserve proposed, blocked, reviewed, and executed actions.

6. Execute three paired six-task sequences with order A/B, B/A, A/B and fresh conversations at each task boundary.

7. Invoke maintenance through the controlled clock at fixed checkpoints and separately verify one real scheduler invocation.

8. Run acceptance and regression checks, score memory outcomes, audit denominators and missing data, and preserve all unfavorable runs.

9. Compute descriptive paired comparisons, component rates, cost breakdowns, and evidence-linked failure traces.

10. Apply the predefined decision rules and report the finding as promising, mixed, negative, or inconclusive within the tested setting.

Appendix D Formal Event Families and Coding Sequence

Event family	Purpose	Typical expected handling
Stable project fact	Test promotion to durable knowledge	Project Memory only with sufficient evidence
Task completion and validation	Test evidence capture and status update	Daily Log plus Session update; durable promotion only when justified
Unfinished session state	Test resumable task continuity	Session Memory with next step and blocker
Failed attempt	Test experiential memory	Daily evidence and Session failure record; later recall when relevant
Repeated event	Test idempotency	One canonical write with retry relationship
Conflicting claim	Test escalation and version handling	Review or supported supersession; no silent overwrite
Changed or stale reference	Test drift detection	Revalidate, preserve history, update or mark uncertain
Explicit user correction	Test authoritative correction with provenance	Versioned update linked to correction event
Synthetic sensitive string	Test secret protection	Block durable write and record safe policy outcome
Irrelevant event	Test selective capture	Skip without creating memory noise

The six connected coding tasks are defined in Table 9. Their order is fixed because later tasks depend on decisions, failures, unfinished work, and repository changes introduced earlier. Each paired repetition begins from the same initial code and memory fixtures, while state is allowed to accumulate within the sequence.